# Gemini Live Agent Hub — Full Research, Plan & Judging Critique

> **Hackathon**: Gemini Live Agent Challenge ($80K prizes)
> **Category**: Live Agents — Real-time Interaction (Audio/Vision)
> **Deadline**: March 17, 2026
> **Target Score**: 6.0 / 6.0 (5.0 scored + 1.0 bonus)

---

## Table of Contents


1. Project Concept
2. Google ADK Research
3. Gemini Live API Research
   - Interleaved Output — Image Generation in Live Sessions
   - Client-Side SDK Research — gemini-live-sdk (npm)
4. E2B Sandbox Research
5. MCP Ecosystem Research
6. WebMCP & Chrome AI Research
   - Chrome Extension — Voice-Activated Tasks
7. Trending MCPs & Skills
8. Desktop Client — Computer Use Agent
   - How Claude Desktop / Computer Use Works
   - Implementation for Agent Hub — Python Tray App
   - Desktop Client Use Cases
   - Reference: UFO$^3$ by Microsoft
9. Architecture Design
   - System Architecture
   - ADK Multi-Agent Architecture
   - TaskArchitect — Dynamic Meta-Orchestrator
   - GenUI — Generative UI on Dashboard
10. Implementation Plan
11. Tech Stack — Rapid Prototyping & Deployment
12. Judging Criteria Alignment
13. Judging Criteria Critique — What's Missing & Risks
14. Enhancement Recommendations per Judging Criterion
15. Scope & Decisions
16. Verification Checklist


---

# 1. Project Concept

## One-Liner

A **multi-client, single-server AI agent hub** where users connect via eyeglasses, web dashboard, or mobile app — all managed by a centralized backend with live voice/video streaming, dynamic MCP plugin system, agent personas with distinct voices, and E2B sandboxed code execution.

## Problem Statement

Current AI assistants are **single-device, single-modality, and static**. You talk to ChatGPT in a text box on one screen. You can't seamlessly transition from your phone to your glasses to your desktop. You can't add new capabilities on the fly. You can't have different "experts" with different voices and knowledge.

## Solution

**Agent Hub** — a unified AI agent backend that:

- Connects to **multiple client types simultaneously** (web dashboard, mobile PWA, ESP32 smart glasses)
- Enables **cross-client actions** ("Hey glasses, save a note to my web dashboard")
- Provides **agent personas** with distinct voices, skills, and instructions (Gemini native audio per-agent voice config)
- Offers an **MCP plugin store** where users add/remove capabilities like installing apps
- Runs code safely in **E2B sandboxes** with 100+ built-in MCP servers
- Uses **proactive audio** and **affective dialog** for truly natural conversation

## Why This Wins

| Differentiator | Why Judges Care |
|---|---|
| Multi-client hub (not just "another chat UI") | "Beyond text" factor — breaks the single-device paradigm |
| Cross-client actions via agent tool calls | Novel interaction pattern never seen before |
| Per-agent voice personas | Live Agents criterion: "distinct persona/voice" |
| MCP plugin store (user-expandable) | Shows system extensibility and real-world value |
| Proactive audio + affective dialog | Demonstrates cutting-edge Gemini native audio features |
| ESP32 glasses support | Hardware + software = immersive multimodal experience |

---

# 2. Google ADK Research

## What is ADK?

**Agent Development Kit** — Google's production-ready framework for building AI agents. Python-first (also TypeScript, Go, Java). `pip install google-adk`.

## Agent Types

| Type | Class | Purpose |
|---|---|---|
| **LLM Agent** | `Agent` / `LlmAgent` | Core agent powered by Gemini. Has instruction, tools, sub_agents |
| **Sequential Agent** | `SequentialAgent` | Runs sub-agents in order |
| **Parallel Agent** | `ParallelAgent` | Runs sub-agents concurrently |
| **Loop Agent** | `LoopAgent` | Repeats sub-agents until condition met |
| **Custom Agent** | `BaseAgent` | Full control, override `_run_async_impl()` |

## Core ADK Components

```
Agent(name, model, instruction, tools, sub_agents)
Runner(app_name, agent, session_service, memory_service, plugins)
SessionService → InMemorySessionService / DatabaseSessionService / VertexAiSessionService
MemoryService → InMemoryMemoryService / VertexAiMemoryBankService
LiveRequestQueue → send_content(), send_realtime(), send_activity_start/end(), close()
RunConfig(streaming_mode, response_modalities, speech_config, proactivity, enable_affective_dialog)
```

## MCP Integration in ADK

```python
from google.adk.tools.mcp_tool import McpToolset
from google.adk.tools.mcp_tool.mcp_session_manager import SseConnectionParams

# Example: Connect to a remote MCP server
mcp_tools = McpToolset(
    connection_params=SseConnectionParams(url="https://mcp-server.example.com/sse")
)

# Or via Stdio (local process)
from google.adk.tools.mcp_tool.mcp_session_manager import StdioConnectionParams
mcp_tools = McpToolset(
    connection_params=StdioConnectionParams(command="npx", args=["-y", "@anthropic/mcp-brave-search"])
)
```

- Supports: `StdioConnectionParams`, `SseConnectionParams`, `StreamableHTTPConnectionParams`
- Features: auth support, tool filtering, header providers, session management

## Skills System (Experimental, ADK v1.25.0+)

```
from google.adk.skills import load_skill_from_dir
from google.adk.tools import skill_toolset

weather_skill = load_skill_from_dir(pathlib.Path(__file__).parent / "skills" / "weather_skill")
my_skill_toolset = skill_toolset.SkillToolset(skills=[weather_skill])

agent = Agent(model="gemini-2.5-flash", tools=[my_skill_toolset])
```

- Skills follow the Agent Skill specification (agentskills.io)
- Structure: `SKILL.md` (required) + `references/` + `assets/` + `scripts/`
- Loaded on-demand to minimize context window impact

## Plugin System (ADK v1.7.0+)

- Plugins extend `BasePlugin`, register on `Runner` via `plugins=[]`
- Global callbacks: `before_agent`, `after_agent`, `before_model`, `after_model`, `before_tool`, `after_tool`, `on_event`, `on_model_error`, `on_tool_error`
- Pre-built: Reflect & Retry Tools, BigQuery Analytics, Context Filter, Global Instruction, Logging
- Plugins run BEFORE agent-level callbacks (precedence)

## Memory System

- `InMemoryMemoryService` — keyword matching, no persistence (dev)
- `VertexAiMemoryBankService` — LLM-powered memory extraction, semantic search (prod)
- Tools: `PreloadMemoryTool` (always load), `LoadMemory` (agent decides)
- Memory workflow: `add_session_to_memory(session)` → `search_memory(query)`

## Session Management

- `Session` = single conversation thread with event history + state
- `State` = key-value data within a session
- `Memory` = cross-session searchable knowledge
- Services: `InMemorySessionService` (dev), `DatabaseSessionService` (SQLite/PostgreSQL/MySQL), `VertexAiSessionService` (managed)

## Callbacks System

- **Agent lifecycle**: `before_agent_callback`, `after_agent_callback`
- **Model lifecycle**: `before_model_callback`, `after_model_callback`
- **Tool lifecycle**: `before_tool_callback`, `after_tool_callback`
- Use for: guardrails, logging, state modification, response caching

### Deployment Options

| Platform | Best For |
|---|---|
| **Cloud Run** | WebSocket/Live agents, auto-scaling, recommended for streaming |
| **Agent Engine (Vertex AI)** | Managed hosting, enterprise features |
| **GKE** | Full Kubernetes control |
| **CLI** | `adk web` for local dev, `adk run` for CLI testing |

### A2A Protocol

- Agent-to-Agent communication protocol
- Allows ADK agents to communicate with agents built on other frameworks
- Out of scope for MVP but architecturally supported

---

## 3. Gemini Live API Research

### What is the Live API?

WebSocket-based bidirectional streaming with Gemini models. Supports continuous audio, video, and text in real-time.

### Technical Specifications

| Spec | Value |
|---|---|
| Audio Input | 16-bit PCM, 16kHz, mono |
| Audio Output | 16-bit PCM, 24kHz, mono (native audio models) |
| Video Input | JPEG, 1fps recommended, 768×768 px |
| Context Window | 32k-128k tokens (model-dependent) |
| Languages | 24+ with automatic detection |
| Session Duration (Gemini API) | Audio-only: 15 min, Audio+video: 2 min (unlimited with context compression) |
| Session Duration (Vertex AI) | 10 min (unlimited with context compression) |
| Concurrent Sessions | Tier-based (Gemini) / Up to 1,000 (Vertex AI) |

## Audio Model Architectures

| Architecture | Model | Features |
|---|---|---|
| Native Audio | `gemini-2.5-flash-native-audio-preview-12-2025` (Gemini API) / `gemini-live-2.5-flash-native-audio` (Vertex AI) | End-to-end audio, natural prosody, extended voice library, affective dialog, proactive audio, automatic language detection |
| Half-Cascade | `gemini-2.0-flash-live-001` (deprecated) / `gemini-live-2.5-flash` (Vertex AI, private GA) | Hybrid: native audio input + TTS output, TEXT modality support, explicit language control |

## Key Live API Features

- **Voice Activity Detection (VAD)**: Auto-detects speech start/end, manages turn-taking
- **Barge-in / Interruption**: Users interrupt mid-response, agent stops and addresses new input
- **Audio Transcription**: Built-in input/output transcription, auto-enabled for multi-agent
- **Session Resumption**: `SessionResumptionConfig()` — reconnect after disconnection
- **Proactive Audio**: Agent initiates responses without explicit prompts (native audio only)
- **Affective Dialog**: Agent detects emotional cues in voice tone, adapts response style (native audio only)

## Voice Configuration

```
from google.adk.models.google_llm import Gemini
from google.genai import types

custom_llm = Gemini(
    model="gemini-2.5-flash-native-audio-preview-12-2025",
    speech_config=types.SpeechConfig(
        voice_config=types.VoiceConfig(
            prebuilt_voice_config=types.PrebuiltVoiceConfig(voice_name="Aoede")
        ),
        language_code="en-US"
    )
)

agent = Agent(name="assistant", model=custom_llm, tools=[...], instruction="...")
```

**Available Voices (Half-cascade)**: Puck, Charon, Kore, Fenrir, Aoede, Leda, Orus, Zephyr
**Native Audio Models**: All above + extended TTS voice library

## Per-Agent Voice in Multi-Agent

```
# Each sub-agent gets its own voice
customer_service = Agent(name="service", model=Gemini(model=MODEL, speech_config=SpeechConfig(voice_name="Aoede")))
tech_support = Agent(name="tech", model=Gemini(model=MODEL, speech_config=SpeechConfig(voice_name="Charon")))
root = Agent(name="root", sub_agents=[customer_service, tech_support])
```

- Multi-agent auto-enables transcription for agent transfer context

## ADK Bidi Streaming Application Lifecycle

```
Phase 1: App Init (once) → Agent, SessionService, Runner
Phase 2: Session Init (per connection) → get/create Session, RunConfig, LiveRequestQueue
Phase 3: Bidi-streaming → upstream (WebSocket→Queue) + downstream (run_live()→WebSocket) via asyncio.gather()
Phase 4: Terminate → LiveRequestQueue.close()
```

## Complete FastAPI Pattern (from bidi-demo)

```python
import asyncio
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from google.adk.runners import Runner
from google.adk.agents.run_config import RunConfig, StreamingMode
from google.adk.agents.live_request_queue import LiveRequestQueue
from google.adk.sessions import InMemorySessionService
from google.genai import types

app = FastAPI()
session_service = InMemorySessionService()
runner = Runner(app_name="agent-hub", agent=agent, session_service=session_service)

@app.websocket("/ws/{user_id}/{session_id}")
async def websocket_endpoint(websocket: WebSocket, user_id: str, session_id: str):
    await websocket.accept()

    run_config = RunConfig(
        streaming_mode=StreamingMode.BIDI,
        response_modalities=["AUDIO"],
        input_audio_transcription=types.AudioTranscriptionConfig(),
        output_audio_transcription=types.AudioTranscriptionConfig(),
        session_resumption=types.SessionResumptionConfig(),
        proactivity=types.ProactivityConfig(proactive_audio=True),
        enable_affective_dialog=True,
    )

    session = await session_service.get_session(app_name="agent-hub", user_id=user_id, session_id=session_id)
    if not session:
        await session_service.create_session(app_name="agent-hub", user_id=user_id, session_id=session_id)

    live_request_queue = LiveRequestQueue()

    async def upstream():
        try:
            while True:
                data = await websocket.receive_bytes()  # Binary PCM audio
                audio_blob = types.Blob(mime_type="audio/pcm;rate=16000", data=data)
                live_request_queue.send_realtime(audio_blob)
        except WebSocketDisconnect:
            pass

    async def downstream():
        async for event in runner.run_live(
            user_id=user_id, session_id=session_id,
            live_request_queue=live_request_queue, run_config=run_config
        ):
            await websocket.send_text(event.model_dump_json(exclude_none=True, by_alias=True))

    try:
        await asyncio.gather(upstream(), downstream(), return_exceptions=True)
    finally:
        live_request_queue.close()
```

## Client-Side Audio (Browser)

- **Capture**: AudioContext(sampleRate: 16000) → AudioWorklet → Float32→PCM16 → binary WebSocket
- **Playback**: AudioContext(sampleRate: 24000) → AudioWorklet ring buffer (180s) → Int16→Float32 → speakers
- **Camera**: getUserMedia(video: 768×768) → canvas.drawImage → toBlob('image/jpeg', 0.85) → base64 → JSON WebSocket
- **Transcription**: `event.inputTranscription` / `event.outputTranscription` with `.text` and `.finished` fields

## Interleaved Output — Image Generation in Live Sessions

### What Is Interleaved Output?

Gemini's Nano Banana image models (`gemini-3.1-flash-image-preview`, `gemini-3-pro-image-preview`, `gemini-2.5-flash-image`) support **interleaved output** — a single response that alternates between text parts and image parts naturally. Setting `response_modalities=['TEXT', 'IMAGE']` returns rich mixed content.

**Supported modes:**

| Mode | Input | Output | Example |
|---|---|---|---|
| Text → Text + Images | Text only | Alternating text + inline images | "Generate an illustrated recipe for paella" |
| Text + Images → Text + Images | Text + image(s) | Alternating text + inline images | *(with room photo)* "What other color sofas would work?" |

**Example response structure (macaron baking guide):**

```
[
  {"text": "### Step 1: Piping the Batter\nThe first step..."},
  {"inline_data": {"data": "<image_data>", "mime_type": "image/png"}},
  {"text": "### Step 2: Baking and Developing Feet\nOnce piped..."},
  {"inline_data": {"data": "<image_data>", "mime_type": "image/png"}},
  {"text": "### Step 3: Assembling the Macaron\nThe final step..."},
  {"inline_data": {"data": "<image_data>", "mime_type": "image/png"}}
]
```

### Critical Limitation — Live API Is Single-Modality Output

> "You can only set one response modality (TEXT or AUDIO) per session." — Gemini Live API docs

The Live API cannot return interleaved text+image. It's either TEXT or AUDIO per session. This means **Nano Banana interleaved output is not available inside `run_live()` sessions directly**.

### Architecture Solution — Image Generation as a Tool, Not a Sub-Agent

Since `run_live()` locks all agents in the tree to the same Live API session & model, a sub-agent **cannot** independently call Nano Banana. Instead, image generation works as a **tool** (a Python function the live agent calls):

```
Live Agent (voice, run_live())
│
├── speaks to user via audio
├── calls tools when needed:
│   ├── generate_image_tool()     ← calls Nano Banana API separately
│   ├── search_tool()
│   ├── code_exec_tool()
│   └── ...
│
│   generate_image_tool() internally:
│     1. Calls client.models.generate_content(model="gemini-3.1-flash-image-preview")
│     2. Gets back interleaved text + image parts
│     3. Saves image to Firestore/GCS (session media)
│     4. Pushes image to dashboard via WebSocket
│     5. Returns TEXT ONLY back to the live agent
│        → "Generated a chart showing 15% growth. Image sent to dashboard."
│     6. Live agent SPEAKS this text description to the user
```

### Why tool, not sub-agent:

| Approach | Works? | Reason |
|---|---|---|
| Sub-agent in same `run_live()` tree | **No** | Sub-agents share the Live API session — can't switch to Nano Banana mid-stream |
| Separate agent microservice | Yes but complex | Requires inter-service comms, adds latency |
| **Tool that calls Nano Banana API** | **Best** | Tool is a Python function — can call any API/model independently, returns text to live agent |

## Implementation

```python
async def generate_image(prompt: str, session_id: str) -> str:
    """Generate an image based on the prompt. Image is sent to user's dashboard."""
    client = genai.Client()

    response = client.models.generate_content(
        model="gemini-3.1-flash-image-preview",
        contents=[prompt],
        config=types.GenerateContentConfig(
            response_modalities=['TEXT', 'IMAGE'],  # interleaved output!
        )
    )

    text_parts = []
    image_count = 0

    for part in response.parts:
        if part.text:
            text_parts.append(part.text)
        elif part.inline_data:
            image_count += 1
            # Save to session storage
            await save_image_to_session(session_id, part.inline_data.data, part.inline_data.mime_type)
            # Push to dashboard via WebSocket
            await push_to_dashboard(session_id, {
                "type": "image",
                "data": base64.b64encode(part.inline_data.data).decode(),
                "mime_type": part.inline_data.mime_type,
                "description": prompt
            })

    summary = "\n".join(text_parts) if text_parts else f"Generated {image_count} image(s) for: {prompt}"
    return f"{summary}\n\nThe image has been sent to your dashboard."

# Register as tool on the live agent
root_agent = Agent(
    name="hub",
    model="gemini-2.5-flash-native-audio-preview-12-2025",
    instruction="When the user asks for visual content, use generate_image tool. "
                "Tell the user to check their dashboard for the image.",
    tools=[generate_image, ...]
)
```

## User Experience Flow

```
User (voice): "Show me a chart of Tesla's stock performance this year"

1. Live agent understands intent → calls generate_image(prompt="Tesla stock chart 2026 YTD")
2. Tool calls Nano Banana → gets chart image + text explanation (interleaved)
3. Tool pushes image to dashboard via WebSocket → user sees it appear on screen
4. Tool saves image to session DB (Firestore/GCS) → persists in conversation history
5. Tool returns text → live agent SPEAKS:
   "I've generated a Tesla stock chart. It shows 15% growth year-to-date
    with a dip in February. Check your dashboard for the visual."
```

## Rich Interleaved Content on Dashboard

For complex requests, Nano Banana returns **multi-step illustrated content** that renders beautifully on the dashboard:

```
User: "Explain photosynthesis with illustrations"

Tool calls Nano Banana → gets back:
  - Text: "Step 1: Light absorption by chlorophyll..."
  - Image: [diagram of chloroplast]
  - Text: "Step 2: Water splitting (photolysis)..."
  - Image: [diagram of H2O molecule splitting]
  - Text: "Step 3: Sugar synthesis (Calvin cycle)..."
  - Image: [diagram of glucose formation]

Dashboard: Renders full illustrated guide (text + images interleaved)
Live Agent speaks: "I've created an illustrated guide on photosynthesis with
                    3 diagrams. Check your dashboard for the visuals."
Session DB: Stores all images + text for history replay
```

### Hackathon Advantage

This dual-channel approach (voice + visual) is a **strong differentiator**:

- Most entries use audio-only OR text-only
- Agent Hub speaks explanations via Live API while simultaneously generating rich visual content on the dashboard
- It showcases Nano Banana (interleaved output) + Live API (native audio) + multi-client architecture in one experience
- The session DB stores images alongside conversation history, so users can review generated content later

---

## Client-Side SDK Research — `gemini-live-sdk` (npm)

> **Package**: `gemini-live-sdk` v1.1.3 · MIT · 365 KB unpacked · 0 weekly downloads
> **Repo**: `github.com/omanandswami2005/gemini-live-sdk`
> **Published**: ~9 months ago (June 2025) · 5 versions · 1 contributor
> **Verdict**: **Extract audio modules only** — transport layer (socket.io) incompatible with our custom Python/FastAPI backend

### Architecture Overview

```
gemini-live-sdk/
├── core/                # GeminiLiveClient — Main class (socket.io based)
│   ├── gemini-client.js  # EventEmitter3 + socket.io-client transport
│   └── types.js          # JSDoc type defs
├── audio/
│   ├── audio-recorder.js # Mic → AudioWorklet → PCM16 → base64 (16kHz)
│   ├── audio-streamer.js # base64 → PCM16 → Float32 → AudioBufferQueue (24kHz)
│   └── worklets/
│       └── audio-recording-worklet.js  # AudioWorkletProcessor (Int16 buffer)
├── media/
│   └── media-handler.js  # Webcam + screen capture → base64 JPEG frames
├── react/
│   ├── hooks/use-gemini-live.js        # React hook wrapping GeminiLiveClient
│   └── components/GeminiLiveProvider.jsx # Context provider
├── server/
│   └── gemini-server.js  # Node.js WebSocket server (NOT relevant for us)
└── utils/
    ├── audio-utils.js    # createAudioContext(), base64⇔ArrayBuffer
    ├── volume-meter.js   # AnalyserNode-based volume visualization
    └── worklet-registry.js # Blob URL worklet loader
```

### Transport Layer — socket.io (❌ Incompatible)

The SDK uses **socket.io-client** (NOT raw WebSocket). Our backend uses FastAPI with raw WebSockets. Socket.io has a custom protocol layer (packet framing, event namespacing, heartbeat) that raw WS servers don't understand.

```
// SDK's transport — socket.io
this.socket = io(this.config.endpoint, {
  auth: { token: this.config.token },
  transports: ['websocket'],
  reconnection: true,
  reconnectionAttempts: this.config.reconnectAttempts,
  reconnectionDelay: this.config.reconnectDelay,
  timeout: 20000
});

// Our backend — raw WebSocket
const ws = new WebSocket(`wss://host/ws/live/${sessionId}`);
```

**Conclusion**: Cannot use `GeminiLiveClient` directly. Must write our own WebSocket transport layer.

### Audio Recorder (✅ Reusable Pattern)

The `AudioRecorder` class uses a clean AudioWorklet pattern worth replicating:

```
Browser Mic → getUserMedia() → MediaStreamSource → AudioWorkletNode → PCM16 chunks → base64 → WebSocket
```

**Key implementation details**:

- **Sample rate**: 16kHz (matches Gemini Live API input requirement)
- **Buffer size**: 2048 Int16 samples (~128ms per chunk at 16kHz)
- **Processing**: Float32 → Int16 conversion in AudioWorklet (off-main-thread)
- **Output**: base64-encoded PCM16 chunks emitted via `data` event
- **Mute**: Implemented by disconnecting source from worklet node (no audio data sent)

```
// AudioWorklet code (runs on audio thread, not main thread)
class AudioProcessingWorklet extends AudioWorkletProcessor {
  constructor() {
    super();
    this.buffer = new Int16Array(2048);
    this.bufferWriteIndex = 0;
  }
  process(inputs) {
    const channel0 = inputs[0][0];
    for (let i = 0; i < channel0.length; i++) {
      this.buffer[this.bufferWriteIndex++] = Math.max(-32768, Math.min(32767, channel0[i] * 32768));
      if (this.bufferWriteIndex >= this.buffer.length) {
        this.port.postMessage({ event: "chunk", data: { int16arrayBuffer: this.buffer.slice(0, this.bufferWriteIndex).buffer } });
        this.bufferWriteIndex = 0;
      }
    }
    return true;
  }
}
```

**Worklet loading** uses a clever Blob URL approach to avoid separate file hosting:

```
const blob = new Blob([workletSourceCode], { type: 'application/javascript' });
const workletUrl = URL.createObjectURL(blob);
await audioContext.audioWorklet.addModule(workletUrl);
```

### Audio Streamer / Playback (✅ Reusable Pattern)

Queue-based playback system for streaming PCM16 audio from Gemini:

```
WebSocket → base64 → ArrayBuffer → PCM16 → Float32 → AudioBuffer → BufferSource → GainNode → Speakers
```

**Key implementation details**:

- **Sample rate**: 24kHz (Gemini Live API output)
- **Queue**: `audioQueue[]` of AudioBuffers — ensures gapless playback
- **Stall detection**: 1-second timeout restarts playback if queue has data but nothing is playing
- **Barge-in**: `stop()` clears queue + ramps gain to 0 in 100ms (smooth cutoff)
- **Resume**: Handles `AudioContext.state === 'suspended'` (browser autoplay policy)
- **GainNode**: Used for volume control + AI volume meter visualization

```
// PCM16 → Float32 conversion
convertPCM16ToFloat32(chunk) {
  const float32 = new Float32Array(chunk.length / 2);
  const view = new DataView(chunk.buffer);
  for (let i = 0; i < chunk.length / 2; i++) {
    float32[i] = view.getInt16(i * 2, true) / 32768; // little-endian
  }
  return float32;
}
```

### Message Protocol (Valuable Reference)

The SDK reveals the Gemini Live API's actual message format:

**Client → Server (audio)**:

```
{
  "realtime_input": {
    "media_chunks": [{ "mime_type": "audio/pcm", "data": "<base64_pcm16>" }]
  }
}
```

**Client → Server (text)**:

```
{
  "client_content": {
    "turns": [{ "role": "user", "parts": [{ "text": "Hello" }] }],
    "turn_complete": true
  }
}
```

**Client → Server (video frame)**:

```
{
  "realtime_input": {
    "media_chunks": [{ "mime_type": "image/jpeg", "data": "<base64_jpeg>" }]
  }
}
```

### Client → Server (tool response):

```
{
  "tool_response": {
    "function_responses": [{ "name": "fn_name", "response": { "result": "..." } }]
  }
}
```

### Server → Client (audio):

```
{
  "serverContent": {
    "modelTurn": {
      "parts": [{ "inlineData": { "data": "<base64_pcm16_24khz>" } }]
    }
  }
}
```

### Server → Client (turn complete / barge-in):

```
{ "serverContent": { "turnComplete": true } }
{ "serverContent": { "interrupted": true } }
```

### Server → Client (transcription):

```
{ "serverContent": { "outputTranscription": { "text": "AI said..." } } }
{ "serverContent": { "inputTranscription": { "text": "User said..." } } }
```

## React Hook Pattern (✅ Good Reference, Need Custom Version)

The `useGeminiLive` hook pattern is well-structured but tightly coupled to `GeminiLiveClient` (socket.io):

```
// What we'll adapt from the SDK's React pattern:
const {
  connectionState,    // { status: 'disconnected'|'connected'|'error' }
  isRecording,        // boolean
  isMuted,            // boolean
  error,              // string | null
  transcriptions,     // [{ type: 'ai'|'user', text, timestamp }]
  webcamActive,       // boolean
  screenActive,       // boolean
  // Methods
  startRecording, stopRecording, toggleMute,
  sendTextMessage, sendToolResponse,
  toggleWebcam, toggleScreenShare,
  createUserVolumeMeter, createAIVolumeMeter,
} = useGeminiLive(config);
```

### Our custom hook will differ:

- Raw `WebSocket` transport instead of socket.io
- Firebase Auth token in first WS message (not socket.io `auth` handshake)
- GenUI event handling (SDK has no concept of this)
- Persona switching via control messages
- Cross-client events from backend
- Zustand store integration instead of local useState

### What to Extract vs Build Custom

| Module | Action | Reason |
|---|---|---|
| `AudioRecorder` | **Extract & adapt** | Solid AudioWorklet pattern, just change output from socket.io emit to raw WS binary |
| `AudioStreamer` | **Extract & adapt** | Good queue + stall detection. Change input from socket.io event to raw WS `onmessage` |
| `AudioWorklet code` | **Copy directly** | Pure audio processing, no transport dependency |
| `audio-utils.js` | **Copy directly** | `createAudioContext()`, `base64ToArrayBuffer()`, `arrayBufferToBase64()` — pure utility |
| `worklet-registry.js` | **Copy directly** | Blob URL worklet loader — 5 lines, elegant |
| `volume-meter.js` | **Extract & adapt** | AnalyserNode visualization — useful for Voice Orb |
| `media-handler.js` | **Extract & adapt** | Webcam + screen capture to base64 JPEG frames |
| `GeminiLiveClient` | ❌ **Rewrite** | socket.io transport incompatible with our FastAPI raw WS |
| `useGeminiLive` hook | ❌ **Rewrite** | Tightly coupled to GeminiLiveClient, need Zustand + raw WS |
| `GeminiLiveProvider` | ❌ **Skip** | We don't need a context provider; Zustand handles global state |
| `GeminiLiveServer` | ❌ **Skip** | Completely replaced by Python/FastAPI + ADK backend |

### Key Differences: SDK Server vs Our Backend

| Aspect | SDK's Node.js Server | Our Python/FastAPI Backend |
|---|---|---|
| Transport | socket.io (Node.js) | Raw WebSocket (FastAPI) |
| AI Framework | Direct Gemini API calls | Google ADK (agents, tools, sessions) |
| Multi-agent | None | Root → Persona sub-agents, TaskArchitect |
| Tool execution | Server-side function declarations | ADK tools + MCP + E2B sandbox |
| Auth | JWT middleware on socket.io | Firebase Auth token verification |
| Events | socket.io event names | JSON message types over raw WS |
| Transcription | Google's built-in transcription events | ADK's transcription via `run_live()` |
| Model | `gemini-2.0-flash-exp` (old) | `gemini-2.5-flash-native-audio-preview` |

### Implementation Plan for Dashboard Audio

Based on SDK patterns, our dashboard audio pipeline will be:

```
┌─ Recording Pipeline ──────────────────────────────────────┐
│ Browser Mic                                                 │
│   → getUserMedia({ audio: true })                           │
│   → AudioContext (16kHz)                                    │
│   → MediaStreamSource                                       │
│   → AudioWorkletNode (Float32 → Int16, 2048-sample buffer)  │
│   → base64-encode PCM16 chunks                              │
│   → WebSocket.send(JSON { type: 'audio', data: base64 })    │
└─────────────────────────────────────────────────────────────┘

┌─ Playback Pipeline ───────────────────────────────────────┐
│ WebSocket.onmessage                                         │
│   → Parse JSON { type: 'response', audio: base64 }          │
│   → base64 → ArrayBuffer → PCM16 → Float32                  │
│   → AudioBuffer (24kHz)                                     │
│   → audioQueue[] (FIFO buffer)                              │
│   → BufferSource.start() → GainNode → speakers              │
│   → Stall detection (1s timeout, auto-restart)              │
│   → Barge-in: stop() → clear queue → gain ramp 0            │
└─────────────────────────────────────────────────────────────┘

┌─ Video/Screen Pipeline ───────────────────────────────────┐
│ getUserMedia / getDisplayMedia                              │
│   → canvas.drawImage(video)                                 │
│   → canvas.toDataURL('image/jpeg', 0.8)                     │
│   → WebSocket.send(JSON { type: 'image', data: base64 })    │
│   → Frame rate: ~1 FPS (configurable)                       │
└─────────────────────────────────────────────────────────────┘
```

### Binary vs JSON Audio Transport Decision

The SDK sends audio as **base64 inside JSON** (via socket.io events). For our raw WebSocket implementation, we have two options:

| Approach | Pros | Cons |
|---|---|---|
| **Binary frames** (raw PCM) | ~33% smaller, no encode/decode overhead, lower latency | Can't mix with JSON in same frame, need frame-type detection |
| **Base64 in JSON** (SDK approach) | Self-describing messages, easy to parse, works with JSON protocol | 33% size overhead, encode/decode CPU cost |

**Decision**: Use **binary WebSocket frames for audio** (both directions) and **JSON text frames for everything else** (text, control, GenUI, transcription). The browser WebSocket API natively distinguishes binary vs text frames via `event.data instanceof Blob`.

```
// Our approach — mixed binary + JSON
ws.onmessage = (event) => {
  if (event.data instanceof Blob) {
    // Binary frame → PCM audio → playback pipeline
    playbackQueue.enqueue(event.data);
  } else {
    // Text frame → JSON message
    const msg = JSON.parse(event.data);
    handleMessage(msg);
  }
};

// Sending audio — binary for performance
ws.send(pcmInt16ArrayBuffer);  // Raw binary, no base64

// Sending text/control — JSON
ws.send(JSON.stringify({ type: 'text', content: 'Hello' }));
```

This gives us the best of both worlds: low-latency audio AND structured JSON for control messages.

---

## 4. E2B Sandbox Research

### What is E2B?

**Open-source secure Linux VMs** (Firecracker microVMs) for AI agents. <200ms cold start, up to 24h sessions.

## Key Features

| Feature | Detail |
|---|---|
| **Cold Start** | <200ms |
| **Max Session** | 24h (Pro) / 1h (Hobby) |
| **Concurrency** | 20 (Hobby) / 100 (Pro) |
| **SDK** | Python (`from e2b import Sandbox`) + JavaScript |
| **Capabilities** | Code execution, terminal commands, file operations, internet access, browser use |
| **MCP Gateway** | Built-in MCP server with 100+ pre-configured MCPs |
| **Computer Use** | Desktop Sandbox for visual UI interaction |
| **Users** | Perplexity, Hugging Face, Manus, Groq |

## Pricing

| vCPUs | Cost |
|---|---|
| 1 | $0.000014/s |
| 2 (default) | $0.000028/s |
| 4 | $0.000056/s |
| 8 | $0.000112/s |

- **Hobby tier**: Free + $100 credits, 1h max session, 20 concurrent
- **Pro tier**: $150/mo + usage, 24h max session, 100 concurrent

## Core API

```
from e2b_code_interpreter import Sandbox

# Create sandbox
sandbox = Sandbox()

# Execute Python
execution = sandbox.run_code('print("hello")')
print(execution.logs)

# Run terminal command
result = sandbox.commands.run("ls -la")

# File operations
sandbox.files.write("/tmp/test.txt", "content")
content = sandbox.files.read("/tmp/test.txt")
files = sandbox.files.list("/")

# Lifecycle
await sandbox.set_timeout(30_000)  # Extend timeout
info = await sandbox.get_info()     # Get sandbox info
await sandbox.kill()                # Shutdown
```

### E2B Built-in MCP Gateway (100+ MCPs)

```
// Create sandbox with MCP servers auto-started
const sandbox = await Sandbox.create({
    mcp: {
        notion: { internalIntegrationToken: "..." },
        context7: {},  // No config needed
        playwright: {},
        braveSearch: { apiKey: "..." },
        codeInterpreter: {},
        github: { personalAccessToken: "..." },
        mongodb: { connectionString: "..." },
        wolframAlpha: { appId: "..." },
        wikipedia: {},
        googleCalendar: { credentials: "..." },
    }
});

const mcpUrl = sandbox.getMcpUrl();
const mcpToken = sandbox.getMcpToken();
```

**Notable built-in MCPs**: notion, context7, github (archived + chat), mongodb, neo4jMemory, playwright, braveSearch, cloudRun, codeInterpreter, nodeCodeSandbox, airtable, atlassian, obsidian, wolframAlpha, wikipedia, googleCalendar, slack, linear, stripe, twilio, sendGrid, and 80+ more.

### Integration with ADK

```
# 1. Create E2B sandbox with MCP gateway
sandbox = Sandbox.create(mcp={"braveSearch": {"apiKey": "..."}})

# 2. Get MCP endpoint
mcp_url = sandbox.get_mcp_url()
mcp_token = sandbox.get_mcp_token()

# 3. Connect ADK McpToolset to E2B's MCP gateway
from google.adk.tools.mcp_tool import McpToolset
from google.adk.tools.mcp_tool.mcp_session_manager import StreamableHTTPConnectionParams

e2b_mcp = McpToolset(
    connection_params=StreamableHTTPConnectionParams(
        url=mcp_url,
        headers={"Authorization": f"Bearer {mcp_token}"}
    )
)

agent = Agent(name="coder", tools=[e2b_mcp], instruction="...")
```

---

## 5. MCP Ecosystem Research

### What is MCP?

**Model Context Protocol** — open standard (by Anthropic) for connecting AI models to external tools and data sources. ADK has native support via `McpToolset`.

### Connection Types in ADK

| Type | Use Case | Class |
|---|---|---|
| **Stdio** | Local process (CLI tools) | `StdioConnectionParams(command, args)` |
| **SSE** | Remote server (Server-Sent Events) | `SseConnectionParams(url)` |
| **StreamableHTTP** | Remote server (HTTP streams) | `StreamableHTTPConnectionParams(url, headers)` |

### MCP Architecture in Our Project

```
User Dashboard → Enable/Disable MCPs → Firestore Config
                                        ↓
Session Start → MCPManager reads config → Build McpToolset[] → Inject into Agent tools
                                        ↓
Agent uses tools → McpToolset routes to MCP server → Result back to agent
```

---

## 6. WebMCP & Chrome AI Research

> **Status**: Early Preview Program (Feb 10, 2026) — bleeding-edge Chrome APIs that can be a **massive hackathon differentiator**

### 6.1 What is WebMCP?

WebMCP is Chrome's upcoming browser-native API that makes websites **"agent-ready"** — enabling AI agents to perform actions on any website with increased speed, reliability, and precision compared to raw DOM scraping/actuation.

**Source**: developer.chrome.com/blog/webmcp-epp — by André Cipriani Bandarra (Google), Feb 10, 2026

#### Two New APIs

| API | Type | How It Works | Use Case |
|---|---|---|---|
| **Declarative API** | HTML forms | Standard actions exposed via annotated HTML forms — browsers/agents discover them automatically | Simple structured actions: submit ticket, search products, fill checkout |
| **Imperative API** | JavaScript | Complex/dynamic interactions registered via JS — enables multi-step workflows | Dynamic filtering, SPA-style interactions, conditional flows |

#### Key Quotes from the Announcement

- *"WebMCP aims to provide a standard way for exposing structured tools, ensuring AI agents can perform actions on your site with increased speed, reliability, and precision"*
- *"These APIs serve as a bridge, making your website 'agent-ready' and enabling more reliable and performant agent workflows compared to raw DOM actuation"*

### Example Use Cases (from Google)

- **Customer Support**: Agent creates support tickets via Declarative API form annotations
- **E-commerce**: Agent searches, filters, and completes checkout via Imperative API
- **Travel Booking**: Agent searches and books flights through structured tool definitions

## 6.2 Chrome DevTools MCP Server

**Package**: `chrome-devtools-mcp` (npm) — **v0.14.0** (latest as of Feb 2026)
**Repo**: ChromeDevTools/chrome-devtools-mcp

This is a **production-grade MCP server** that wraps Chrome DevTools Protocol, giving AI agents full browser control.

### 29+ Tools Across 7 Categories

| Category | Tools | What They Do |
|---|---|---|
| **Input Automation** (9) | `click`, `drag`, `fill`, `fill_form`, `handle_dialog`, `hover`, `press_key`, `type_text`, `upload_file` | Full user interaction simulation |
| **Navigation** (6) | `close_page`, `list_pages`, `navigate_page`, `new_page`, `select_page`, `wait_for` | Page management |
| **Emulation** (2) | `emulate` (geolocation, network, CPU, user agent), `resize_page` | Device/network simulation |
| **Performance** (4) | `performance_analyze_insight`, `performance_start_trace`, `performance_stop_trace`, `take_memory_snapshot` | Performance profiling |
| **Network** (2) | `get_network_request`, `list_network_requests` | Network monitoring |
| **Debugging** (6) | `evaluate_script`, `get_console_message`, `lighthouse_audit`, `list_console_messages`, `take_screenshot`, `take_snapshot` | Page debugging & screenshots |
| **Extensions** (4) | `install_extension`, `uninstall_extension`, `list_extensions`, `reload_extension` | Extension management |

### Key Features

- `--auto-connect` flag: auto-discover and attach to running Chrome instance
- `--slim` mode: Only 3 tools (navigate, evaluate, screenshot) — lightweight for simple use
- `--experimental-vision`: Computer vision support for visual page understanding
- Preserved logs across navigations
- Compatible with: Gemini CLI, Claude, Cursor, Copilot, Windsurf, Factory CLI

## 6.3 Prompt API Function Calling (On-Device AI)

**Source**: Chromium Intent to Prototype (blink-dev, Aug 7, 2025) — by Sushanth Rajasankar (Microsoft) + Google engineers

Adds **function calling / tool use** to Chrome's on-device **Prompt API** (powered by Gemini Nano running locally in the browser).

### Two Complementary APIs ("Two sides of the same coin")

| API | Direction | Who Controls | Purpose |
|---|---|---|---|
| **Prompt API Function Calling** | Inward-facing | Web page is orchestrator | Page enhances its own features — page defines tools, on-device LLM invokes them |
| **Script Tools API** | Outward-facing | External agent controls page | External agents (browser-level, OS-level) discover and invoke capabilities exposed by web pages |

### Why Not Just MCP Directly?

Google/Microsoft considered exposing MCP directly to the web but chose a higher-level native API instead:

- *"We believe a higher-level, native API is better suited for the web platform"*
- Approach is "**isomorphic with concepts in MCP but abstracts away low-level protocol details**"
- Advantages: Direct DOM access, developer ergonomics, durability (not coupled to evolving MCP spec), **offline/privacy** (on-device Gemini Nano, no network needed)

## 6.4 How to Leverage WebMCP for the Hackathon

### Strategy 1: Make the Web Dashboard "Agent-Ready" with WebMCP

```
Web Dashboard (React)
  ├─ Declarative API: Annotate dashboard forms (MCP config, persona settings)
  │   → External agents can configure the hub without DOM scraping
  ├─ Imperative API: Register complex actions via JS
  │   → "enable_mcp", "switch_persona", "start_session", "view_transcript"
  └─ Result: Our own dashboard becomes an MCP-compatible tool that OTHER agents can use
```

**Why this matters**: Instead of just USING MCPs, our project BECOMES an MCP server that any AI agent can interact with. This is a meta-level innovation.

### Strategy 2: Chrome DevTools MCP as a Plugin

```
Plugin Store → "Chrome Browser Control" plugin
  → Uses chrome-devtools-mcp
  → Agent can: navigate websites, fill forms, take screenshots, run Lighthouse audits
  → Demo: "Hey agent, open the competitor's website and run a performance audit"
```

### Strategy 3: On-Device AI via Prompt API in Chrome Extension Client

```
Chrome Extension Client (future client type)
  ├─ Uses Prompt API Function Calling with Gemini Nano
  ├─ Runs LOCALLY in browser — no server round-trip for simple tasks
  ├─ Defines Script Tools so the main Agent Hub server can invoke page actions
  └─ Hybrid architecture: Simple queries → on-device, Complex queries → server
```

### Strategy 4: Demo Wow-Factor

- Show the dashboard's WebMCP Declarative API → an external Claude/Gemini agent can **control our own dashboard** via MCP
- Live demo: "Agent A uses chrome-devtools-mcp to scrape a website → pipes data to Agent B for analysis → results displayed on dashboard"
- Mention: *"Our architecture is WebMCP-ready — when Chrome ships this API in stable, our hub is automatically agent-accessible"*

### Judging Impact

| Criterion | WebMCP Boost |
|---|---|
| **Innovation** | Using bleeding-edge Chrome APIs no other hackathon entry will use |
| **Technical Depth** | Multi-layer: WebMCP Declarative + Imperative + DevTools MCP + Prompt API |
| **Future-Proofing** | Architecture designed for the next generation of web-AI interaction |
| **Demo Polish** | Chrome DevTools MCP = visual, impressive browser automation in demo video |

## 6.5 Chrome Extension — Voice-Activated Tasks

With the Chrome extension client connected to Agent Hub, users can control their browser entirely by voice. The extension captures voice via the browser microphone, streams it to the Agent Hub backend, and the agent executes browser actions via Chrome DevTools MCP or extension APIs.

### Tab & Navigation

| Voice Command | What Happens |
|---|---|
| "Open a new tab" | Opens blank tab |
| "Go to Gmail" | Navigates to gmail.com |
| "Close this tab" | Closes active tab |
| "Switch to tab 3" | Activates the 3rd tab |
| "Go back" / "Go forward" | Browser nav history |
| "Bookmark this page" | Adds to bookmarks |
| "Open my bookmarks for AI research" | Opens bookmark folder |
| "Reopen closed tab" | Restores last closed tab |

### Search & Browse

| Voice Command | What Happens |
|---|---|
| "Search for latest Gemini API updates" | Opens Google search |
| "Find on this page: pricing" | Ctrl+F and searches for "pricing" |
| "Summarize this page" | Agent reads page content (via DevTools MCP `evaluate_script` or DOM extraction) and returns summary |
| "Read this article to me" | Agent extracts article text, speaks it via Live API audio |
| "What is this page about?" | Agent screenshots page → Gemini Vision analysis |
| "Save this page as PDF" | Triggers print-to-PDF |

### Email (Gmail)

| Voice Command | What Happens |
|---|---|
| "Compose email to John about the meeting" | Opens Gmail compose, fills recipient + subject + draft body |
| "Read my latest email" | Navigates to inbox, extracts top email, reads aloud |
| "Reply saying I'll be there at 3pm" | Opens reply, types response |
| "Archive this email" | Clicks archive button |
| "Search emails from Sarah last week" | Uses Gmail search bar |
| "Mark all as read" | Bulk action on inbox |

### Calendar (Google Calendar)

| Voice Command | What Happens |
|---|---|
| "What's on my schedule today?" | Opens Google Calendar, reads events aloud |
| "Create a meeting with Alex at 3pm tomorrow" | Opens event creation form, fills details |
| "Reschedule my 2pm to 4pm" | Finds event, drags or edits time |
| "What's my next meeting?" | Reads the upcoming event |

### Content Interaction

| Voice Command | What Happens |
|---|---|
| "Fill this form with my info" | Agent reads form fields, fills with stored user profile data |
| "Download this file" | Clicks download link |
| "Screenshot this page" | Uses DevTools MCP `take_screenshot` → saves/sends to dashboard |
| "Translate this page to Spanish" | Triggers Chrome translate or Google Translate |
| "Extract all links from this page" | DOM extraction → returns link list |
| "Copy the main content" | Extracts article body to clipboard |

### Productivity

| Voice Command | What Happens |
|---|---|
| "Save this to Notion" | Extracts page content → sends to Notion MCP |
| "Create a task in Linear: review API docs" | Calls Linear MCP to create task |
| "Add to my reading list" | Saves URL to Notion/bookmarks/reading list |
| "Start a 25-minute focus timer" | Sets browser notification timer |
| "Take a note: remember to check pricing page" | Saves note to session DB |

### Accessibility

| Voice Command | What Happens |
|---|---|
| "Zoom in" / "Zoom out" | Adjusts page zoom level |
| "Enable dark mode" | Toggles dark reader / prefers-color-scheme |
| "Read this aloud" | TTS of selected or page content |
| "Increase font size" | Adjusts minimum font via extension CSS injection |
| "High contrast mode" | Applies accessibility overlay |

### Developer Tools

| Voice Command | What Happens |
|---|---|
| "Open DevTools" | Opens Chrome DevTools panel |
| "Run a Lighthouse audit" | Uses DevTools MCP `lighthouse_audit` → reports scores |
| "Show me network requests" | Uses DevTools MCP `list_network_requests` |
| "Clear cache and hard reload" | Cache clear + reload |
| "What JavaScript errors are on this page?" | Uses DevTools MCP `list_console_messages` filtered to errors |
| "Take a performance trace" | Uses DevTools MCP `performance_start_trace` / `performance_stop_trace` |

### Agent Hub–Specific

| Voice Command | What Happens |
|---|---|
| "Switch to Coder persona" | Sends persona switch command to backend |
| "Enable Brave Search plugin" | Toggles MCP in user config |
| "Show my dashboard" | Opens Agent Hub web dashboard in new tab |
| "What clients are connected?" | Queries Client Registry, reads aloud |
| "Send this page summary to my phone" | Cross-client action → pushes to mobile client |
| "Ask Sage to explain this page" | Delegates to Researcher persona with page context |

### Why Voice-Activated Browser Matters

- **Hands-free browsing** — ideal when multitasking, cooking, exercising
- **Accessibility** — enables users with motor impairments to fully control the browser
- **Speed** — voice commands can be faster than mouse+keyboard for routine actions
- **Cross-client synergy** — say a command on glasses → executes on browser; results appear on dashboard
- **Hackathon differentiator** — most entries are text-only; voice-controlled browser is visually impressive in demos

---

## 7. Trending MCPs & Skills

### High-Value MCPs (Recommended for Plugin Store)

| MCP | Category | Why Trending | Demo Value |
|---|---|---|---|
| **Brave Search** | Web Search | Privacy-focused, widely adopted in AI agents | "Search the web for..." — instant demo |
| **Playwright** | Browser Automation | Hot trend: AI agents interacting with websites | "Open this webpage and extract..." |
| **Context7** | Dev Documentation | Real-time library docs lookup | "Look up the React useState docs" |
| **GitHub** | Developer Tools | Universal developer tool | "Create an issue on my repo" |
| **Notion** | Productivity | Popular knowledge management | "Save this note to my Notion" |
| **MongoDB** | Database | Data persistence for agents | "Store this data, query it later" |
| **Google Calendar** | Scheduling | Practical daily use case | "What's on my calendar today?" |
| **Wolfram Alpha** | Computation | Math / science problem solving | "Calculate the integral of..." |
| **Wikipedia** | Knowledge | Instant knowledge lookup | "Tell me about quantum computing" |
| **Filesystem** | Local Files | Desktop client file management | "Read the file at ~/documents/report.pdf" |
| **Slack** | Communication | Team messaging integration | "Send a message to #general" |
| **Linear** | Project Management | Engineering team workflow | "Create a task in Linear" |

## Skills to Define (via ADK Skills spec / agentskills.io)

| Skill | Description | MCPs Used |
|---|---|---|
| **Code Review** | Analyzes code quality, suggests improvements, runs tests | E2B code interpreter |
| **Research Assistant** | Compiles findings from multiple sources | Brave Search + Wikipedia |
| **Meeting Prep** | Briefs user on upcoming meetings with context | Google Calendar + Notion |
| **Visual Analysis** | Processes camera input, describes scenes | Camera + Google Search |
| **Data Analyst** | Queries databases, generates charts | MongoDB + E2B (matplotlib) |

## Non-Developer, Professional-Use Trending MCPs & Skills

These are the MCPs and skills that bring **real value to professionals outside of software engineering** — doctors, lawyers, marketers, educators, finance people, content creators, operations managers, and more. This is a **massive hackathon differentiator**: most AI agent projects target developers only. Building an agent hub that serves everyday professionals = wider real-world impact = higher judging scores.

### Healthcare & Biomedical

| MCP / Skill | What It Does | Professional Who Uses It |
|---|---|---|
| **BioMCP** | Access PubMed, ClinicalTrials.gov, MyVariant.info for biomedical research | Doctors, Clinical Researchers, Pharma teams |
| **DGIdb** | Drug-gene interaction lookups, druggable genome info, pharmacogenomics | Pharmacists, Drug Discovery researchers |
| **FHIR** | Healthcare data interoperability standard access | Health IT professionals, Clinical data analysts |
| **Dicom** | Query/retrieve medical images, parse DICOM-encapsulated documents | Radiologists, Medical Imaging techs |
| **Open Targets** | Target-disease associations, drug discovery data | Biomedical researchers, Pharma |
| **FitBit / Oura Ring** | Personal health data — sleep, HR, activity | Fitness coaches, Wellness professionals |

### Finance & Business

| MCP / Skill | What It Does | Professional Who Uses It |
|---|---|---|
| **Financial Datasets** | Stock market data — income statements, balance sheets, cash flow | Financial Analysts, Investors |
| **Nasdaq Data Link** | Extensive financial and economic datasets | Quant researchers, Economists |
| **FRED** | Federal Reserve economic data | Economists, Policy analysts |
| **QuantConnect** | Algorithmic trading — design, backtest, deploy strategies | Quant traders, Portfolio managers |
| **Morningstar** | Research, editorial data, investment datapoints | Investment advisors, Wealth managers |
| **Stripe / Razorpay / Mercado Pago** | Payment processing — invoicing, customer management | Business owners, Freelancers |
| **HubSpot** | CRM integration — contacts, companies, pipelines | Sales teams, Account managers |
| **Salesforce** | Full CRM interactions with Salesforce | Enterprise sales, CRM admins |
| **Ramp** | Corporate spend analysis & insights | Finance ops, CFOs |
| **Windsor** | Full-stack business data integration & analysis | Business analysts, Data-driven managers |

### Legal & Compliance

| MCP / Skill | What It Does | Professional Who Uses It |
|---|---|---|
| **Brazilian Law** | Agent-driven research on Brazilian law via official sources | Lawyers (Brazil-focused), Compliance |
| **Congress.gov API** | Real-time US Congressional data, bills, committees | Policy analysts, Legal researchers |
| **Companies House MCP** | UK company registry lookups (KRS, filings) | Corporate lawyers, Due diligence |
| **Drata** | Real-time compliance intelligence into AI workflows | Compliance officers, GRC teams |
| **SEC EDGAR (USPTO)** | Patent & trademark data, financial filings | IP attorneys, Patent analysts |
| **mcp-sanctions** | Screen individuals/orgs against global sanctions lists (OFAC, UN) | AML/KYC teams, Compliance officers |

### Marketing & Content Creation

| MCP / Skill | What It Does | Professional Who Uses It |
|---|---|---|
| **Metricool** | Social media analytics — performance metrics & scheduling across platforms | Social media managers, Marketers |
| **Facebook Ads / TikTok Ads / Amazon Ads** | Campaign management, performance analytics, audience targeting | Digital marketers, Media buyers |
| **SEO MCP (kwrds.ai)** | Keyword research, SERP analysis, backlinks (Ahrefs data) | SEO specialists, Content strategists |
| **KeywordsPeopleUse** | Find questions people ask online — content ideation | Content creators, SEO writers |
| **Plus AI / 2slides / Slidespeak** | Automated PowerPoint/Google Slides creation from prompts | Marketers, Sales, Consultants (anyone making decks) |
| **Canva / Placid.app** | Template-based image and video creative generation | Graphic designers, Social media teams |
| **Substack/Medium** | Semantic search and analysis of published content | Writers, Newsletter operators |
| **LinkedIn MCP** | Write, edit, schedule LinkedIn posts | Personal branders, B2B marketers |
| **WaveSpeed / Fal AI** | AI image & video generation for campaigns | Creative directors, Content teams |
| **Open Strategy Partners** | Content editing codes, value map, positioning tools | Product marketing managers |

### Education & Research

| MCP / Skill | What It Does | Professional Who Uses It |
|---|---|---|
| **Rember** | Spaced repetition flashcard creation from AI chats | Students, Teachers, Self-learners |
| **Anki** | Full Anki deck management — create, update, search cards | Students, Medical residents, Language learners |
| **Scholarly / OpenReview** | Search academic articles, fetch AI/ML conference papers | Researchers, PhD students |
| **bioRxiv** | Search and access bioRxiv preprints | Life science researchers |
| **OpenAlex** | ML-powered author disambiguation, researcher profiles | Academic researchers, Librarians |
| **Zettelkasten** | AI-powered knowledge management — atomic notes, link discovery | Researchers, Knowledge workers |
| **Wolfram Alpha** | Computational math & science problem solving | STEM educators, Students |
| **mcp-open-library** | Search books and author info via Open Library | Librarians, Readers, Educators |

### Productivity & Office Automation

| MCP / Skill | What It Does | Professional Who Uses It |
|---|---|---|
| **Microsoft 365 (Lokka)** | Full M365 — Teams, SharePoint, Exchange, OneDrive, Entra, Intune | Enterprise workers (everyone) |
| **Google Workspace CLI** | Gmail, Calendar, Drive, Sheets from a single interface | Anyone using Google Workspace |
| **Notion** | Knowledge base — search, create, update pages/databases | PMs, Ops managers, Content teams |
| **Airtable** | Read/write access to Airtable databases | Ops managers, Marketers, Project leads |
| **Monday.com** | Board and item management | Project managers, Team leads |
| **Todoist / TickTick** | Task management and to-do list | Anyone managing tasks |
| **Slack** | Channel management and messaging | Team communication (universal) |
| **Obsidian Notes** | Note vault search, reading, writing, organizing | Knowledge workers, Writers |
| **Trello** | Board modification via prompts | Small team project management |
| **Excel File Manipulation** | Read/write Excel without installing MS Office | Data entry, Accountants, Ops |
| **Pandoc / Markdownify** | Document format conversion (PDF, DOCX, HTML, PPTX → Markdown) | Document-heavy professionals |
| **Office-Word / PowerPoint / Visio** | Create, read, manipulate MS Office documents | Enterprise workers, Consultants |
| **Inbox Zero** | AI email assistant for reaching inbox zero | Anyone drowning in email |

### Travel & Lifestyle

| MCP / Skill | What It Does | Professional Who Uses It |
|---|---|---|
| **Airbnb** | Search listings, get details | Travel planners, Remote workers |
| **Tripadvisor** | Location data, reviews, photos | Travel agents, Hospitality |
| **Foursquare** | Place recommendations worldwide | Marketers (location data), Travelers |
| **TomTom** | Geospatial mapping, routing | Logistics teams, Delivery ops |
| **Google Maps** | Location services, directions, place details | Universal |
| **Duffel (Flights)** | Personalized flight recommendations | Travel agents, Corporate travel |
| **Amadeus** | Flight offers search — airlines, times, duration, pricing | Travel industry professionals |
| **Triplyfy** | Itinerary planning with interactive maps | Travel bloggers, Trip planners |

### Real Estate & Property

| MCP / Skill | What It Does | Professional Who Uses It |
|---|---|---|
| **Zillow / Redfin (via web scraping)** | Property search, home valuations | Real estate agents, Home buyers |
| **FDIC BankFind** | US banking data — structured bank information | Mortgage brokers, Financial advisors |

### IoT & Smart Home

| MCP / Skill | What It Does | Professional Who Uses It |
|---|---|---|
| **Home Assistant MCP** | Control 2000+ device brands (Hue, Tuya, Sonoff, etc.) via one API — `call_service`, `get_state`, `list_entities`, `fire_event` | Smart home users, Facility managers |
| **LG ThinQ Connect** | Control LG smart home devices & appliances | Smart home enthusiasts, Facility managers |
| **Tuya MCP** | Budget smart plugs, bulbs, sensors (Tuya/Smart Life ecosystem) via Tuya Cloud API | Budget IoT users |
| **MQTT MCP** | Generic MQTT client — any MQTT device (ESP32, Zigbee bridges, custom sensors) | DIY IoT builders, Industrial IoT |
| **Philips Hue MCP** | Hue lights, motion sensors, switches via Bridge API | Smart lighting users |
| **ESPHome MCP** | Custom ESP32/ESP8266 devices via native API — synergizes with our ESP32 glasses hardware | Makers, IoT hobbyists |
| **Apple Script** | Full Mac automation via LLM | Power users, IT admins |
| **Siri Shortcuts** | Interact with all Siri Shortcuts on macOS | Mac power users |
| **iMCP (iMessage, Reminders)** | Apple services integration | Anyone in Apple ecosystem |

> **ESP32 Glasses × IoT Synergy**: Glasses camera detects dark room → agent proactively offers to turn on lights via Home Assistant MCP → voice confirms → lights on. This showcases **vision + voice + IoT** in one loop — a strong "beyond text" demo moment.

### Media & Creative

| MCP / Skill | What It Does | Professional Who Uses It |
|---|---|---|
| **Blender** | AI-assisted 3D modeling, scene creation | 3D artists, Game designers |
| **Unity / Godot** | Game engine integration | Game developers, XR creators |
| **Ableton Live** | Music creation with prompt-assisted production | Musicians, Producers |
| **Fish Audio** | Text-to-Speech with multiple voices, real-time playback | Podcasters, Voiceover artists |
| **Video Jungle** | Add, edit, search video content | Video editors, Content creators |
| **YouTube (yutu)** | Full YouTube automation — upload, manage | YouTubers, Social media managers |
| **ZapCap** | Video caption and B-roll generation | Video editors, Accessibility |
| **Figma** | AI-driven design operations, asset management | UI/UX designers |

## Why This Matters for the Hackathon

1. **Wider Real-World Impact** — Judges score "real-world problem solving." An agent hub that serves doctors, lawyers, marketers, and students — not just developers — demonstrates **massive applicability**
2. **Plugin Store Showcase** — The MCP plugin store becomes 10x more impressive with professional verticals: "Browse by: Healthcare | Finance | Marketing | Education | Productivity"
3. **Demo Storytelling** — Instead of just "here's a coding assistant," demo scripts can show: *"A financial analyst asks the glasses to check today's market data and FRED economic indicators while walking to a meeting..."*
4. **Persona × MCP Synergy** — Each agent persona (Doctor, Lawyer, Analyst, Educator) comes pre-configured with relevant MCPs. This is a novel UX pattern
5. **Judging Edge on Innovation** — Most hackathon entries focus on dev tools. Showing non-dev professional use cases = **unique angle** that judges rarely see

### Recommended Demo Verticals (Pick 2-3 for Live Demo)

| Vertical | Persona | MCPs Used | Demo Script |
|---|---|---|---|
| **Finance Analyst** | "Nova" (analytical voice) | Financial Datasets + FRED + E2B (charts) | "Nova, show me Apple's latest quarterly revenue compared to last year. Plot it." |
| **Healthcare Researcher** | "Dr. Atlas" (calm, authoritative) | BioMCP + PubMed + DGIdb | "Dr. Atlas, what are the latest clinical trials for GLP-1 receptor agonists?" |
| **Marketing Manager** | "Spark" (energetic voice) | Metricool + SEO MCP + LinkedIn | "Spark, how's our Instagram engagement this week? Draft a LinkedIn post about it." |
| **Student / Researcher** | "Sage" (patient, educational) | Scholarly + Wolfram + Rember | "Sage, explain Fourier transforms and create flashcards for my exam." |
| **Executive Assistant** | "Claire" (professional, warm) | Google Calendar + M365 + Slack + Notion | "Claire, what's on my schedule today? Summarize the Notion page from Monday's strategy meeting and post a recap to #team-updates." |

---

## 8. Desktop Client — Computer Use Agent

> **Decision**: If built, the desktop client will be a **lightweight Python tray app** (not Electron) — minimal UI footprint, maximum automation power. It connects to the same Agent Hub backend as web/mobile/glasses.

### How Claude Desktop / Computer Use Works

Claude's "computer use" capability is the current state-of-the-art for desktop AI agents. Understanding how it works informs our own implementation.

### The Agent Loop

```
User Request (e.g., "Open Excel and create a budget spreadsheet")
       ↓
┌──────────────────────┐
│  1. Take Screenshot  │ ← Capture current display (Xvfb / native screen)
└──────────────────────┘
          │
          ↓
┌──────────────────────┐
│  2. Send to Vision   │ ← Screenshot → multimodal LLM for analysis
│     Model (Gemini)   │    "What do you see? What UI elements are present?"
└──────────────────────┘
          │
          ↓
┌──────────────────────┐
│  3. Decide Action    │ ← LLM reasons: "I need to click the Start menu"
│     (LLM Reasoning)  │    Returns structured action: {click: [x, y]}
└──────────────────────┘
          │
          ↓
┌──────────────────────┐
│  4. Execute Action   │ ← pyautogui.click(x, y) / typewrite(text) / hotkey()
└──────────────────────┘
          │
          ↓
┌──────────────────────┐
│  5. Wait + Re-check  │ ← Brief pause for UI to update, then loop to step 1
└──────────────────────┘
```

This is identical to Claude's `computer_20251124` tool — a closed loop: **Look → Reason → Act → Look (repeat)**.

### Claude's Available Actions

| Action | Description |
|---|---|
| `screenshot` | Capture current display |
| `left_click` / `right_click` / `middle_click` | Click at coordinates [x, y] |
| `double_click` / `triple_click` | Multi-click at coordinates |
| `type` | Type text string |
| `key` | Press key combo (e.g., "ctrl+s") |
| `mouse_move` | Move cursor to coordinates |
| `scroll` | Scroll in any direction with amount control |
| `left_click_drag` | Click and drag between coordinates |
| `hold_key` | Hold a key for N seconds |
| `wait` | Pause between actions |
| `zoom` | View specific screen region at full resolution (Opus 4.6+) |

Claude augments this with a **bash tool** (run terminal commands) and **text editor tool** (read/write files) for more comprehensive automation.

### Key Limitations (Applies to Any Desktop Agent)

| Limitation | Impact | Our Mitigation |
|---|---|---|
| **Latency** | Each look→reason→act cycle takes 1-3s | Pre-plan multi-step actions, batch when possible |
| **Vision accuracy** | LLM may misidentify UI elements or coordinates | Use pywin32 UIA APIs for precise element targeting when available |
| **Scrolling reliability** | Scroll amounts can be imprecise | Use keyboard shortcuts (Page Down, arrow keys) as fallback |
| **Multi-monitor** | Hard to reason across multiple displays | Limit to primary display initially |
| **Security** | Agent could be tricked by on-screen prompt injections | Sandboxed actions, user confirmation for destructive operations |

## Implementation for Agent Hub — Python Tray App

### Architecture

```
┌──────────────────────────────────────────────┐
│              User's Windows Desktop          │
│                                              │
│  ┌──────────────────────────────────────┐    │
│  │  Agent Hub Desktop Client (Python)   │    │
│  │  • System tray icon (pystray)        │    │
│  │  • WebSocket connection to Agent Hub backend │  │
│  │  • Screen capture (pyautogui)        │    │
│  │  • Action execution (pyautogui + pywin32)  │  │
│  │  • File operations (pathlib + shutil)  │  │
│  │  • Process management (psutil + subprocess) │  │
│  │  • Clipboard (pyperclip)             │    │
│  └──────────────┬──────────────────────┘    │
│                 │ WebSocket (wss://)          │
└─────────────────┼────────────────────────────┘
                  ↕
┌──────────────────────────────────────────────┐
│        Agent Hub Backend (Cloud Run)         │
│  ADK Agent with desktop automation tools     │
│  • ScreenCaptureTool → receives screenshot   │
│  • ClickTool(x,y) → sends click command      │
│  • TypeTool(text) → sends keystrokes         │
│  • WindowManagementTool → activate/minimize/close │
│  • FileOperationsTool → list/read/write/delete  │
│  • ProcessManagementTool → launch/kill apps  │
│  • ClipboardTool → copy/paste operations     │
│  • CrossClientActionTool → send commands to other │
│    clients (glasses, web, mobile)            │
└──────────────────────────────────────────────┘
```

## Core Technologies

| Layer | Package | Purpose |
|---|---|---|
| **System Tray** | `pystray` + `Pillow` | Lightweight tray icon, menu, start/stop |
| **Screen Capture** | `pyautogui` | `screenshot()` → PIL Image → base64 → WebSocket |
| **Vision Analysis** | Gemini 2.5 Flash (server-side) | Screenshot sent to backend → Gemini vision analyzes |
| **Mouse/Keyboard** | `pyautogui` + `pywin32` | Click, type, hotkeys, drag |
| **Window Mgmt** | `pygetwindow` + `pywin32` | Activate, minimize, maximize, list windows |
| **File Operations** | `pathlib` + `shutil` | List, read, write, copy, move, delete files |
| **App Launching** | `subprocess` + `psutil` | Launch apps, list/kill processes |
| **Clipboard** | `pyperclip` | Read/write clipboard for cross-app data transfer |
| **System Info** | `psutil` + `screeninfo` | CPU/RAM/disk usage, screen resolution |
| **WebSocket** | `websockets` | Persistent connection to Agent Hub backend |

## Execution Flow Example

```
# Desktop client receives command from backend:
# {"action": "screenshot"}
screenshot = pyautogui.screenshot()
img_bytes = io.BytesIO()
screenshot.save(img_bytes, format='JPEG', quality=85)
await ws.send(base64.b64encode(img_bytes.getvalue()))  # Send to backend

# Backend sends screenshot to Gemini Vision → gets action:
# {"action": "click", "x": 500, "y": 300}
pyautogui.click(500, 300)

# Backend sends type command:
# {"action": "type", "text": "Hello World"}
pyautogui.typewrite('Hello World', interval=0.02)

# Backend sends hotkey:
# {"action": "hotkey", "keys": ["ctrl", "s"]}
pyautogui.hotkey('ctrl', 's')
```

## ADK Tool Definitions (Server-Side)

```python
async def capture_screen(session_id: str) -> str:
    """Request a screenshot from the desktop client. Returns image analysis."""
    screenshot_data = await desktop_clients[session_id].request_screenshot()
    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=["Describe what you see on this desktop screenshot.", screenshot_data]
    )
    return response.text

async def click_at(x: int, y: int, session_id: str) -> str:
    """Click at pixel coordinates on the desktop."""
    await desktop_clients[session_id].send_action({"action": "click", "x": x, "y": y})
    return f"Clicked at ({x}, {y})"

async def type_text(text: str, session_id: str) -> str:
    """Type text on the desktop."""
    await desktop_clients[session_id].send_action({"action": "type", "text": text})
    return f"Typed: {text}"

async def open_application(app_name: str, session_id: str) -> str:
    """Launch an application on the desktop."""
    await desktop_clients[session_id].send_action({"action": "launch", "app": app_name})
    return f"Launched {app_name}"

async def manage_files(operation: str, path: str, dest: str = None, session_id: str = "") -> str:
    """File operations: list, read, write, copy, move, delete."""
    await desktop_clients[session_id].send_action(
        {"action": "file_op", "operation": operation, "path": path, "dest": dest}
    )
    return f"File operation '{operation}' on {path} completed"
```

## Desktop Client Use Cases

| Category | Voice Command Example | What Happens |
|---|---|---|
| **File Management** | "Organize my Downloads folder — move images to Pictures, PDFs to Documents" | Agent screenshots desktop → identifies files → executes move operations via file tools |
| **Multi-App Workflow** | "Copy the Q3 revenue from the Excel file and paste it into the email draft" | Opens Excel → locates cell → copies → switches to email → pastes |
| **Browser Automation** | "Fill out the job application form on this website with my resume info" | Screenshots browser → identifies fields → types info → submits |
| **Code Editing** | "Open VS Code, create a new Python file called utils.py, and add a logging helper" | Launches VS Code → creates file → types code |
| **Document Processing** | "Merge all PDFs in my Reports folder into one file" | Uses subprocess to call a PDF tool or Python library |
| **System Admin** | "Check my disk space and delete temp files if less than 10GB free" | Checks `psutil.disk_usage()` → cleans temp if needed |
| **Calendar/Email** | "Open Outlook and schedule a meeting with the product team for Friday 2pm" | Launches Outlook → navigates to calendar → creates event |
| **Presentation** | "Open PowerPoint, create a 5-slide deck about our Q3 results" | Launches PowerPoint → creates slides with content |
| **Data Entry** | "Enter these 20 customer records from the spreadsheet into the CRM web form" | Reads spreadsheet data → opens CRM → fills forms repeatedly |
| **Cross-Device ★** | "Hey glasses, take a note" → note appears on desktop Notion app | Voice from glasses → backend → desktop client opens Notion → types note |

### The Killer Feature: Cross-Client Desktop Actions

The desktop client's biggest value in the Agent Hub ecosystem is **cross-client synergy**:

```
👓 Glasses (voice): "Save a note about today's meeting decisions"
   ↓ (audio → backend)
🖥️ Desktop: Agent opens Notion → types meeting notes
   ↓ (confirmation → backend)
📱 Mobile: Push notification: "Meeting notes saved to Notion"
   ↓ (sync)
🌐 Dashboard: Note appears in conversation history + session artifacts
```

No other hackathon entry will have:

- Voice on wearable → action on desktop → confirmation on phone → visible on web dashboard
- This is the "**unified AI assistant across all your devices**" story that wins Innovation points

### Reference: UFO³ by Microsoft

**Status**: Production-ready, LTS | **GitHub**: microsoft/UFO (8.1K ⭐, 994 forks) | **Language**: Python

Microsoft's UFO³ Galaxy is the closest open-source equivalent to our desktop client concept:

| Feature | UFO³ Galaxy | Our Agent Hub Desktop Client |
|---|---|---|
| **Multi-device** | ✅ Windows, Linux, Android | ✅ Windows + glasses + mobile + web |
| **Task decomposition** | DAG-based ConstellationAgent | TaskArchitect dynamic pipeline |
| **Communication** | WebSocket AIP protocol | WebSocket to Agent Hub backend |
| **MCP integration** | ✅ Tool augmentation | ✅ Full MCP plugin store |
| **Voice-first** | ❌ Text/UI only | ✅ Gemini Live Audio native |
| **Agent personas** | ❌ Single agent style | ✅ Multiple voices & personalities |
| **Concurrent execution** | ✅ Async parallel | ✅ ParallelAgent via ADK |

**Our competitive advantage over UFO**: Voice-first interaction via Gemini Live Audio, agent personas with distinct voices, multi-client spanning hardware (glasses), user-installable MCP plugin store, and E2B sandbox for safe code execution.

---

# 9. Architecture Design

## System Architecture

```
┌──────────────────────────────────────────────────────────┐
│                   Google Cloud (Cloud Run)                 │
│                                                            │
│  ┌──────────────────────────────────────────────────┐     │
│  │              FastAPI Backend (Python)              │     │
│  │                                                    │     │
│  │   ┌──────────────┐    ┌──────────────────┐        │     │
│  │   │ ADK Runner   │    │ MCPManager       │        │     │
│  │   │ run_live()   │    │ Dynamic McpToolset│       │     │
│  │   │ per session  │    │ per user config  │        │     │
│  │   └──────┬───────┘    └────────┬─────────┘        │     │
│  │          │                     │                  │     │
│  │   ┌──────┴───────┐    ┌────────┴─────────┐        │     │
│  │   │ Root Agent   │    │ E2B Sandbox      │        │     │
│  │   │ (Router)     │    │ Service          │        │     │
│  │   │  ├ Assistant │    │ Code execution   │        │     │
│  │   │  ├ Coder     │    │ MCP gateway      │        │     │
│  │   │  ├ Researcher│    └──────────────────┘        │     │
│  │   │  └ Creative  │                                │     │
│  │   └──────┬───────┘    ┌──────────────────┐        │     │
│  │          │            │ Client Registry  │        │     │
│  │          │            │ Track devices    │        │     │
│  │          │            │ Cross-client cmds │       │     │
│  │          │            └──────────────────┘        │     │
│  │          │                                        │     │
│  │   ┌──────┴───────┐    ┌──────────────────┐        │     │
│  │   │ Persona      │    │ Firestore        │        │     │
│  │   │ Manager      │    │ Sessions, Config │        │     │
│  │   └──────────────┘    │ Personas, MCPs   │        │     │
│  │                       └──────────────────┘        │     │
│  └──────────────────────────────────────────────────┘     │
│                   ↕ WebSocket (wss://)                     │
└──────────────────────────────────────────────────────────┘
          ↕                   ↕                   ↕
┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│ Web Dashboard │    │ Mobile PWA   │    │ ESP32 Glasses│
│ React + TS    │    │ React + TS   │    │ Mic + Speaker│
│ Full dashboard│    │ Voice-first  │    │ + Camera     │
│ MCP store     │    │ Camera       │    │ WiFi→WS      │
│ Persona mgmt  │    │ Responsive   │    │ PCM audio    │
└──────────────┘    └──────────────┘    └──────────────┘
```

## Data Flow

```
1. Client connects: WebSocket → /ws/{user_id}/{session_id}/{client_type}
2. Backend registers client in ClientRegistry
3. Backend loads user's MCP config from Firestore → builds McpToolset[]
4. Backend creates LiveRequestQueue + starts run_live() event loop
5. Client sends audio (binary PCM) → upstream task → LiveRequestQueue.send_realtime()
6. Client sends text (JSON) → upstream task → LiveRequestQueue.send_content()
7. Client sends image (JSON base64) → upstream task → LiveRequestQueue.send_realtime()
8. Agent processes → yields Event[] → downstream task → WebSocket.send_text()
9. Agent calls MCP tool → McpToolset routes → MCP server → result → agent continues
10. Agent calls cross_client_action → ClientRegistry → WebSocket message to target client
11. Client disconnects → LiveRequestQueue.close() → session saved
```

## ADK Multi-Agent Architecture — Leveraging All Agent Types

ADK provides 4 orchestration agent types beyond the basic `LlmAgent`. Using them effectively demonstrates **deep architectural sophistication** — a key judging criterion.

### Agent Type Reference

| Agent Type | Class | Behavior | When to Use |
|---|---|---|---|
| **LlmAgent** | `Agent` / `LlmAgent` | LLM decides what to do — can call tools, delegate to sub_agents | Dynamic routing, conversation, any task needing reasoning |
| **SequentialAgent** | `SequentialAgent` | Runs sub-agents **in strict order**, output of step N flows to step N+1 | Multi-step workflows where each step depends on the previous |
| **ParallelAgent** | `ParallelAgent` | Runs sub-agents **concurrently**, all results collected when done | Independent data gathering from multiple sources — 4x faster |
| **LoopAgent** | `LoopAgent` | Repeats sub-agents **until exit condition met** or max iterations | Iterative refinement — draft→critique→improve cycles |

### Where Each Agent Type Fits in Agent Hub

#### LlmAgent — Root Router + All Specialist Personas

The root agent uses Gemini's natural language understanding to dynamically route to the right specialist. This is **not hardcoded if/else** — the LLM decides based on intent.

```
root_agent = Agent(
    name="hub_router",
    model="gemini-2.5-flash-native-audio-preview-12-2025",
    instruction="You are the Agent Hub router. Analyze user intent and delegate to the best specialist.",
    sub_agents=[assistant, coder, researcher, analyst, creative]
)
```

Each sub-agent persona is also an `LlmAgent` with its own voice, instruction, and MCP tools:

```
coder = Agent(name="coder", instruction="You are a coding expert...",
              tools=[e2b_tools, github_tools], generate_content_config=RunConfig(
                  speech_config=SpeechConfig(voice_config=VoiceConfig(prebuilt_voice_id="Kore"))))
```

### SequentialAgent — Multi-Step Workflows

| User Request | Sequential Pipeline | Why Sequential |
|---|---|---|
| "Research quantum computing and write a report" | `SearchAgent` → `SynthesizeAgent` → `FormatAgent` | Step 2 needs step 1's results, step 3 formats step 2's output |
| "Check calendar, find free slots, draft a meeting invite" | `CalendarReaderAgent` → `SlotFinderAgent` → `EmailDraftAgent` | Each step feeds into the next |
| "Get stock data, analyze trends, create a chart" | `DataFetchAgent` → `AnalysisAgent` → `ChartAgent (E2B)` | Analysis needs data first, chart needs analysis |
| "Analyze this code, run tests, fix bugs" | `CodeAnalyzerAgent` → `TestRunnerAgent` → `BugFixerAgent` | Must analyze → test → then fix |

```
research_pipeline = SequentialAgent(
    name="deep_research",
    sub_agents=[search_agent, synthesize_agent, format_agent]
)
```

### ParallelAgent — Concurrent Data Gathering

| User Request | Parallel Agents | Speed Benefit |
|---|---|---|
| "Brief me for my day" | `CalendarAgent` ‖ `NotionAgent` ‖ `SlackAgent` ‖ `EmailAgent` | 4 sources simultaneously — **4x faster** than sequential |
| "Compare 3 competitor websites" | `WebScraperA` ‖ `WebScraperB` ‖ `WebScraperC` | All 3 scraped at once |
| "What's new today?" (Morning Briefing) | `NewsAgent` ‖ `CalendarAgent` ‖ `StockAgent` ‖ `WeatherAgent` | All fetched in parallel |
| "Health dashboard" | `FitBitAgent` ‖ `OuraAgent` ‖ `CalendarAgent` | Sleep + ring + schedule simultaneously |

```
morning_briefing = ParallelAgent(
    name="morning_briefing",
    sub_agents=[news_agent, calendar_agent, stock_agent, weather_agent]
)
```

### LoopAgent — Iterative Refinement Until Quality Bar Met

| User Request | Loop Pipeline | Exit Condition |
|---|---|---|
| "Write and polish a blog post" | `DraftAgent` → `CriticAgent` → `RewriteAgent` (loop) | Critic scores ≥ 8/10 |
| "Debug this code until tests pass" | `CodeFixAgent` → `TestRunnerAgent` (loop) | All tests green |
| "Refine my presentation" | `SlideGenAgent` → `ReviewAgent` → `ImproveAgent` (loop) | Review approves all slides |
| "SEO optimize this content" | `ContentAgent` → `SEOCheckAgent` → `OptimizeAgent` (loop) | SEO score ≥ 90 |

```
content_refiner = LoopAgent(
    name="content_refiner",
    sub_agents=[draft_agent, critic_agent],
    max_iterations=5
)
```

### Full Agent Tree — Nested Composition

This is the **complete agent hierarchy** using all 4 types in a nested tree:

```
Root Agent "Hub" (LlmAgent — Router)
│
├── "Assistant" (LlmAgent)                      ← Simple Q&A, daily tasks
│   voice: Puck | MCPs: [Google Calendar, Slack, Notion, Gmail]
│
├── "Coder" (LlmAgent)                          ← Code execution & debugging
│   voice: Kore | MCPs: [E2B, GitHub, Filesystem]
│   └── debug_loop (LoopAgent)                   ← Fix → Test → Fix until passing
│       ├── fix_agent (LlmAgent)
│       └── test_agent (LlmAgent + E2B)
│
├── "Researcher" (LlmAgent)                     ← Deep research tasks
│   voice: Aoede | MCPs: [Brave Search, Wikipedia, Scholarly, bioRxiv]
│   └── deep_research (SequentialAgent)          ← Search → Synthesize → Format
│       ├── search_agent (LlmAgent + Brave)
│       ├── synthesize_agent (LlmAgent)
│       └── format_agent (LlmAgent)
│
├── "Analyst" (LlmAgent)                        ← Finance & data analysis
│   voice: Charon | MCPs: [Financial Datasets, FRED, E2B]
│   └── analysis_pipeline (SequentialAgent)      ← Fetch → Analyze → Chart
│       ├── data_fetch (ParallelAgent)           ← Multiple sources at once
│       │   ├── stock_fetcher (LlmAgent)
│       │   └── economic_fetcher (LlmAgent)
│       ├── analyze_agent (LlmAgent)
│       └── chart_agent (LlmAgent + E2B matplotlib)
│
├── "Morning Briefing" (ParallelAgent)           ← All-at-once daily briefing
│   ├── news_agent (LlmAgent + Brave)
│   ├── calendar_agent (LlmAgent + Google Calendar)
│   ├── stock_agent (LlmAgent + Financial Datasets)
│   └── weather_agent (LlmAgent)
│
└── "Creative" (LlmAgent)                       ← Content creation
    voice: Leda | MCPs: [Plus AI Slides, LinkedIn, WaveSpeed]
    └── content_loop (LoopAgent)                 ← Draft → Critique → Refine
        ├── draft_agent (LlmAgent)
        └── critic_agent (LlmAgent)
```

### Architecture Diagram Strategy (for Submission)

Use **color-coded boxes** in the architecture diagram:

- 🔵 **Blue** = `LlmAgent` (decision makers, personas)
- 🟢 **Green** = `SequentialAgent` (ordered pipelines)
- 🟠 **Orange** = `ParallelAgent` (concurrent execution)
- 🟣 **Purple** = `LoopAgent` (iterative refinement)

This single diagram immediately communicates architectural sophistication. Most hackathon entries use a single flat agent — a **nested multi-agent tree** is a fundamentally different level of engineering.

### Judging Criteria Impact of Multi-Agent Architecture

| Architecture Pattern | What Judges See | Score Impact |
|---|---|---|
| **LlmAgent routing** | Dynamic intent understanding — not hardcoded if/else | High — shows LLM-native design |
| **SequentialAgent pipelines** | Sophisticated multi-step reasoning chains | High — shows workflow orchestration |
| **ParallelAgent gathering** | Performance optimization — fast response times | Medium-High — shows engineering quality |
| **LoopAgent refinement** | Self-improving output quality — "agent iterates until good" | High — shows autonomous quality control |
| **Nested composition** | `ParallelAgent` inside `SequentialAgent` inside `LlmAgent` | **Very High** — deep technical architecture |
| **Per-agent voice configs** | Each persona sounds different when speaking | High — live audio differentiation |
| **Per-agent MCP toolsets** | Each persona has domain-specific capabilities | High — shows modular design |

### Key Demo Moments to Showcase Multi-Agent Architecture

1. **"Brief me for my day"** → `ParallelAgent` fires 4 sub-agents simultaneously → results stream back fast → visually impressive speed
2. **"Research quantum computing and write a report"** → `SequentialAgent` executes steps in order → show progress on dashboard (Step 1/3, 2/3, 3/3)
3. **"Write me a LinkedIn post, make it great"** → `LoopAgent` drafts, self-critiques, rewrites → show iteration count on dashboard (Draft 1 → Score: 6/10, Draft 2 → Score: 8/10 ✓)
4. **"Switch to Coder mode"** → Root `LlmAgent` delegates to Coder sub-agent with different voice → voice audibly changes → demonstrates multi-agent delegation live
5. **"Get Apple's revenue data and create a chart"** → `SequentialAgent` with nested `ParallelAgent` for data fetch → shows 2-level nesting in one request

## TaskArchitect — Dynamic Meta-Orchestrator

Beyond predefined multi-agent trees, **TaskArchitect** is a meta-orchestrator that **dynamically analyzes any complex task, decomposes it into sub-tasks, and builds a custom multi-agent pipeline on the fly** — choosing Sequential, Parallel, Loop, or Hybrid patterns per sub-task. The resulting agent team executes autonomously while the user watches live progress on the dashboard.

**Key distinction from Personas:**

- **Personas** (Jarvis, Friday, Sage) = user-created; they define personality, voice, preferred tools
- **TaskArchitect pipelines** = system-created; they are dynamic agent TEAMS assembled per-task, visible on dashboard as a live execution graph

### How It Works

```
User Request (complex task)
       ↓
┌──────────────────────┐
│   TaskArchitect Agent   │  ← Analyzes task complexity, dependencies, parallelizability
│   (Meta-Orchestrator)   │
└──────────────────────┘
       ↓
┌──────────────────────┐
│   Task Decomposition    │  ← Breaks into sub-tasks, identifies dependencies
│   + Pipeline Selection  │  ← Chooses Sequential / Parallel / Loop / Hybrid
└──────────────────────┘
       ↓
┌──────────────────────┐
│  Dynamic Agent Pipeline │  ← Creates & runs the agent team
│  (visible on Dashboard) │  ← Live DAG visualization, status, progress
└──────────────────────┘
       ↓
    Aggregated Result
```

### Real-World Examples

#### Example 1: "Plan my 2-week trip to Japan on a $5K budget"

TaskArchitect analyzes → multi-source research + constraint optimization + sequential dependencies

```
ParallelAgent("Research") [
    FlightSearchAgent      → finds flights, prices, dates
    HotelSearchAgent       → finds accommodations by region
    AttractionAgent        → top attractions per city
    WeatherAgent           → 2-week forecast
    LocalFoodAgent         → restaurant recommendations
    TransitAgent           → JR Pass, local transport
]
    ↓
SequentialAgent("Plan") [
    ItineraryBuilderAgent  → combines parallel results into day-by-day plan
    BudgetOptimizerAgent   → fits within $5K, suggests trade-offs
    BookingPrepAgent       → generates booking links & checklist
]
    ↓
LoopAgent("Refine") [
    UserPreferenceCheckAgent → "too many temples? more food experiences?"
    AdjustmentAgent          → modifies itinerary
    → loops until user approves
]
```

**Dashboard view:** 6 parallel agents running simultaneously (research phase) with progress indicators, then sequential planning pipeline, then refinement loop with iteration counter.

#### Example 2: "Analyze Tesla's financials and write an investment report"

TaskArchitect → data gathering (parallelizable) + analysis (sequential) + quality loop

```
ParallelAgent("DataGather") [
    SECFilingsAgent        → 10-K, 10-Q, earnings transcripts
    MarketDataAgent        → stock price, P/E, market cap trends
    NewsAgent              → recent news, analyst ratings
    CompetitorAgent        → comparison with Rivian, BYD, Ford EV
    MacroEconomicAgent     → EV market trends, policy changes
]
    ↓
SequentialAgent("Analysis") [
    FinancialAnalysisAgent → revenue growth, margins, cash flow, debt
    RiskAssessmentAgent    → identifies key risks & catalysts
    ValuationAgent         → DCF model, comparable analysis
]
    ↓
LoopAgent("Report") [
    ReportWriterAgent      → drafts full investment report
    QualityCheckerAgent    → checks accuracy, completeness, clarity
    → loops until quality_score > 0.9
]
    ↓
FormattingAgent            → PDF with charts, tables, executive summary
```

**Dashboard view:** 5 data gatherers in parallel with progress bars, then analysis pipeline flowing through, then quality loop showing score improving (0.6 → 0.75 → 0.92 ✓).

#### Example 3: "Debug why our production API is returning 500 errors"

TaskArchitect → diagnostic (parallel probing) + correlation + iterative hypothesis testing

```
ParallelAgent("Diagnose") [
    LogAnalyzerAgent       → scans error logs, stack traces
    MetricsAgent           → CPU, memory, disk, request latency
    DatabaseAgent          → slow queries, connection pool status
    NetworkAgent           → DNS, SSL, upstream service health
    DeploymentAgent        → recent deploys, config changes
]
    ↓
CorrelationAgent           → cross-references all findings, identifies patterns
    ↓
LoopAgent("RootCause") [
    HypothesisAgent        → proposes likely root cause
    TestAgent              → runs diagnostic commands/queries to verify
    ValidateAgent          → confirms or rejects hypothesis
    → loops until root cause confirmed
]
    ↓
SequentialAgent("Resolve") [
    FixRecommendationAgent → generates fix with code/config changes
    ImpactAnalysisAgent    → assesses risk of the fix
    IncidentReportAgent    → creates post-mortem document
]
```

**Dashboard view:** 5 diagnostic probes running simultaneously, then correlation analysis, then hypothesis-test loop (Hypothesis 1: DB pool exhausted → Test → Reject → Hypothesis 2: memory leak after v2.3.1 deploy → Test → **Confirmed** ✓).

#### Example 4: "Create a complete marketing campaign for our SaaS product launch"

TaskArchitect → research → strategy (sequential) → content creation (parallel) → quality loop

```
ParallelAgent("Research") [
    CompetitorAnalysisAgent  → competitor messaging, pricing, channels
    AudienceResearchAgent    → ICP, pain points, buying triggers
    TrendAnalysisAgent       → market trends, seasonal timing
    ChannelAnalysisAgent     → best-performing channels for SaaS
]
    ↓
StrategyAgent              → creates campaign strategy, positioning, messaging framework
    ↓
ParallelAgent("Create") [
    AdCopyAgent             → Google Ads, LinkedIn Ads, Facebook Ads copy
    EmailSequenceAgent      → 5-email nurture sequence
    LandingPageAgent        → landing page copy + structure
    SocialContentAgent      → 30-day social calendar with posts
    BlogPostAgent           → 3 launch-related blog posts
    PRAgent                 → press release draft
]
    ↓
LoopAgent("BrandCheck") [
    BrandConsistencyAgent   → checks tone, voice, messaging alignment
    RevisionAgent           → fixes inconsistencies
    → loops until brand_score > 0.95
]
    ↓
BudgetAllocationAgent      → allocates spend across channels with ROI projections
```

**Dashboard view:** Research (4 parallel), strategy (single), then 6 content agents working simultaneously — the "wow" moment where the user sees 6 agents all producing content at once.

#### Example 5: "Prepare me for my Google SWE interview next week"

TaskArchitect → research (parallel) + gap analysis (sequential) + prep (parallel) + mock loop

```
ParallelAgent("Intel") [
    CompanyResearchAgent    → Google culture, recent products, team info
    RoleAnalysisAgent       → SWE L4 requirements, tech stack
    InterviewPatternAgent   → Google's interview format, common patterns
    GlassdoorAgent          → recent interview experiences, questions asked
]
    ↓
GapAnalysisAgent           → compares user's resume/skills vs requirements
    ↓
ParallelAgent("Prep") [
    CodingPrepAgent         → curated LeetCode problems by Google frequency
    SystemDesignAgent       → practice questions (Design YouTube, Gmail)
    BehavioralAgent         → STAR stories framework, culture-fit answers
    QuestionBankAgent       → 50 likely questions with model answers
]
    ↓
LoopAgent("MockInterview") [
    MockInterviewerAgent    → asks questions, evaluates answers
    FeedbackAgent           → scores, identifies weak areas
    FocusedPracticeAgent    → drills weak areas
    → loops for N rounds or until confidence > 0.8
]
    ↓
FinalPrepAgent             → cheat sheet, day-of checklist, confidence boosters
```

### Dashboard Visualization for TaskArchitect Pipelines

| Element | Display |
|---|---|
| **Live Agent DAG** | Visual graph — nodes are agents, edges show data flow. Animated when running |
| **Agent Status** | 🟢 Running (pulse), 🟡 Waiting, ✅ Complete, 🔴 Error |
| **Progress** | Overall % bar + per-agent progress |
| **Live Logs** | Stream of each agent's findings/output in real-time |
| **Timeline** | Gantt-like view showing when each agent ran and duration |
| **Loop Counter** | For LoopAgents: "Iteration 3/max 5 — Quality: 0.82" |
| **Resource Meter** | Tokens used, API calls, E2B sandbox time |
| **Pipeline Blueprint** | The architecture the TaskArchitect chose (shown before execution begins) |

## ADK Implementation Sketch

```python
class TaskArchitectAgent(CustomAgent):
    """Meta-orchestrator that dynamically composes agent pipelines."""

    async def analyze_task(self, task: str) -> PipelineBlueprint:
        # Use Gemini to analyze task complexity and decompose
        analysis = await self.model.generate(f"""
        Analyze this task and decompose it into sub-tasks:
        Task: {task}

        Return JSON:
        {{
          "sub_tasks": [
            {{"id": "t1", "description": "...", "tools_needed": [...]}}
          ],
          "dependencies": {{"t2": ["t1"], "t3": ["t1"]}},
          "parallelizable_groups": [["t2", "t3"], ["t5", "t6"]],
          "requires_iteration": ["t4"],
          "pipeline_type": "hybrid"
        }}
        """)
        return PipelineBlueprint.from_analysis(analysis)

    async def build_pipeline(self, blueprint: PipelineBlueprint) -> Agent:
        # Dynamically construct agent graph from the blueprint
        agents = []
        for stage in blueprint.stages:
            if stage.type == "parallel":
                agents.append(ParallelAgent(
                    name=stage.name,
                    sub_agents=[self._create_agent(t) for t in stage.tasks]
                ))
            elif stage.type == "loop":
                agents.append(LoopAgent(
                    name=stage.name,
                    sub_agents=[self._create_agent(t) for t in stage.tasks],
                    max_iterations=stage.max_iter
                ))
            elif stage.type == "sequential":
                agents.append(SequentialAgent(
                    name=stage.name,
                    sub_agents=[self._create_agent(t) for t in stage.tasks]
                ))
            else:
                agents.append(self._create_agent(stage.tasks[0]))

        return SequentialAgent(name="DynamicPipeline", sub_agents=agents)

    def _create_agent(self, task_spec) -> Agent:
        # Create an LlmAgent tuned for a specific sub-task
        return Agent(
            name=task_spec.agent_name,
            model="gemini-2.5-flash",
            instruction=task_spec.instruction,
            tools=self._resolve_tools(task_spec.tools_needed)
        )

    async def execute(self, task: str):
        blueprint = await self.analyze_task(task)
        # Send blueprint to dashboard via WebSocket for visualization
        await self.broadcast_to_dashboard("pipeline_created", blueprint.to_dict())

        pipeline = await self.build_pipeline(blueprint)
        result = await pipeline.run(callbacks=[DashboardProgressCallback()])
        return result
```

### Implementation Complexity Assessment

| Component | Complexity | Effort | Notes |
|---|---|---|---|
| **Task analysis prompt engineering** | Medium | 1 day | Gemini analyzes tasks well — main work is prompt tuning for reliable JSON output |
| **PipelineBlueprint data model** | Low | 0.5 day | Dataclass with stages, tasks, dependencies — straightforward |
| **Dynamic agent construction** | Medium-High | 1.5 days | Map blueprint stages → ADK agent types, resolve tools per agent, handle edge cases |
| **Dashboard DAG visualization** | High | 2 days | React Flow or D3.js DAG rendering, WebSocket updates for live status, animations |
| **Progress callbacks** | Medium | 1 day | ADK callbacks (before/after_agent_call) → WebSocket events → dashboard updates |
| **Loop exit condition evaluation** | Medium | 1 day | LLM-based quality scoring + threshold checking per LoopAgent |
| **Error handling & fallbacks** | Medium | 1 day | What if a sub-agent fails? Retry? Skip? Replan? Need graceful degradation |
| **Pipeline history & replay** | Low-Medium | 0.5 day | Store completed blueprints in Firestore, show history on dashboard |
| **TOTAL** | | **~8 days** | Can be reduced to **~4 days** by simplifying dashboard viz (no fancy DAG, just a step list) |

### Risk & Mitigation

| Risk | Impact | Mitigation |
|---|---|---|
| Gemini produces bad blueprint JSON | Pipeline won't build | Strict JSON schema validation + retry with corrective prompt |
| Too many sub-agents for simple tasks | Wastes tokens, slow | Complexity threshold — tasks below it skip TaskArchitect, go to regular agent |
| Dashboard viz takes too long to build | Delays hackathon | MVP: simple ordered step list with status icons. Fancy DAG is stretch goal |
| Sub-agent failures cascade | Whole pipeline fails | Wrap each agent in try/catch, allow partial results, report failures in dashboard |

### Judging Criteria Impact

| Criteria | Why TaskArchitect Scores High |
|---|---|
| **Agent Architecture** (highest weight) | Not just multi-agent — an agent that **dynamically creates** multi-agent architectures. Meta-level sophistication. |
| **Innovation** | Nobody else will have an agent that analyzes tasks and auto-composes pipelines. Most entries hard-code their agent graphs. |
| **Visual Demo Impact** | Judges watch the dashboard as a complex task spawns a custom agent team — parallel agents light up simultaneously, sequential ones flow through, loops iterate with improving quality scores. |
| **Practical Value** | Users don't need to know about agent architectures — the system figures it out. True AI autonomy. |

## GenUI — Generative UI on Dashboard

Instead of rendering agent responses as plain text in a chat window, the dashboard can **dynamically generate React UI components** based on tool results — charts, tables, forms, cards, image galleries, interactive widgets.

### What Is GenUI?

Generative UI (GenUI) is a pattern where the AI agent's response determines not just *what* content to show but *how* to display it. The agent returns structured data + a component hint, and the dashboard renders the appropriate React component.

### How It Works in Agent Hub

```
User: "Show me Tesla's stock performance this year"
       ↓
Agent calls financial_data_tool() → returns structured JSON:
{
  "type": "chart",
  "chartType": "line",
  "title": "Tesla (TSLA) YTD Performance",
  "data": [{"date": "2026-01", "price": 392}, ...],
  "summary": "Tesla is up 15% YTD with a February dip..."
}
       ↓
Dashboard receives via WebSocket → matches "type": "chart" → renders <LineChart />
Agent speaks summary via Live API audio simultaneously
```

### Component Mapping

| Agent Response Type | React Component | Example Use Case |
|---|---|---|
| `chart` (line/bar/pie) | `<DynamicChart />` | Stock data, analytics, metrics |
| `table` | `<DataTable />` | Search results, comparisons, logs |
| `card` | `<InfoCard />` | Meeting briefs, person profiles, summaries |
| `code` | `<CodeBlock />` | Code execution results, file contents |
| `image_gallery` | `<ImageGallery />` | Generated images (Nano Banana), screenshots |
| `form` | `<DynamicForm />` | User input collection, settings |
| `timeline` | `<Timeline />` | TaskArchitect pipeline progress, event history |
| `map` | `<MapView />` | Travel planning, location-based results |
| `kanban` | `<KanbanBoard />` | Task management, project status |
| `markdown` | `<MarkdownRenderer />` | Long-form content, reports, documentation |
| `diff` | `<DiffViewer />` | Code changes, document comparisons |
| `weather` | `<WeatherWidget />` | Weather forecasts |

### Implementation Approach

```
// Dashboard component mapper
function AgentResponse({ message }) {
  const data = JSON.parse(message.structuredData);

  switch (data.type) {
    case 'chart':    return <DynamicChart {...data} />;
    case 'table':    return <DataTable {...data} />;
    case 'card':     return <InfoCard {...data} />;
    case 'code':     return <CodeBlock {...data} />;
    case 'image_gallery': return <ImageGallery {...data} />;
    case 'form':     return <DynamicForm {...data} onSubmit={handleFormSubmit} />;
    case 'timeline': return <Timeline {...data} />;
    default:         return <MarkdownRenderer content={message.text} />;
  }
}
```

The agent's tool functions return structured JSON with a `type` field. The backend forwards this via WebSocket. The dashboard maps `type` → React component. This is inspired by Vercel AI SDK's `streamUI` concept but adapted for our WebSocket-first architecture.

### Why GenUI Matters

- **Beyond chat** — transforms the dashboard from a chat window into a **dynamic workspace**
- **Judges see visual richness** — charts, cards, and tables are far more impressive than plain text in a demo video
- **Real utility** — financial analysts want charts, not paragraphs describing numbers
- **Reusable** — same components work for any persona (Analyst shows charts, Coder shows code blocks, Researcher shows tables)
- **Concurrent audio + visual** — agent speaks the summary while the dashboard renders the visual. Dual-channel experience.

---

## 10. Implementation Plan

### Phase 1: Backend Core (Days 1-3) — CRITICAL PATH

#### Step 1.1: Project Scaffolding

```
backend/
  main.py                    # FastAPI app, WebSocket endpoints
  agents/
    __init__.py
    personas.py              # Agent persona definitions
    router_agent.py          # Root agent with sub_agents
  services/
    client_registry.py       # Track connected clients
    mcp_manager.py           # Dynamic MCP toolset loading
    e2b_service.py           # E2B sandbox management
    persona_service.py       # Firestore CRUD for personas
  config.py                  # Env vars, model config
  requirements.txt
  .env
```

**Dependencies**: `google-adk`, `google-genai`, `google-cloud-firestore`, `e2b-code-interpreter`, `fastapi`, `uvicorn`, `python-dotenv`

**Environment**:

```
# Development (Gemini Live API)
GOOGLE_GENAI_USE_VERTEXAI=FALSE
GOOGLE_API_KEY=your_key

# Production (Vertex AI Live API)
GOOGLE_GENAI_USE_VERTEXAI=TRUE
GOOGLE_CLOUD_PROJECT=your_project
GOOGLE_CLOUD_LOCATION=us-central1

E2B_API_KEY=your_e2b_key
```

### Step 1.2: ADK Agent + Live Streaming

- WebSocket endpoint with `client_type` parameter
- Upstream/downstream `asyncio.gather()` pattern
- `RunConfig` with proactive audio + affective dialog
- Model: `gemini-2.5-flash-native-audio-preview-12-2025`

### Step 1.3: Client Registry + Cross-Client Tools

- ADK custom tools: `cross_client_action()`, `list_connected_clients()`
- ClientRegistry tracks `{user_id → {client_id → status}}`

### Step 1.4: Agent Personas

- 4 default personas with unique `Gemini(speech_config=...)` voices
- Root agent delegates to persona sub-agents
- Persona CRUD in Firestore

## Phase 2: MCP Plugin System (Days 3-5)

### Step 2.1: MCPManager

- Dynamic `McpToolset` instantiation from Firestore config
- Support for Stdio, SSE, StreamableHTTP connections
- Pre-configured catalog of 10+ MCPs

### Step 2.2: E2B Integration

- `execute_code()` and `run_terminal_command()` as ADK custom tools
- E2B MCP gateway bridge to ADK via `StreamableHTTPConnectionParams`

### Step 2.3: Google Search Grounding

- ADK built-in `google_search` tool for grounding

## Phase 3: Web Dashboard (Days 4-6)

### Step 3.1: React + TypeScript + Vite + Tailwind

- Chat View, Persona Manager, MCP Store, Client Status, Sandbox Console

### Step 3.2: Browser Audio/Video

- AudioWorklet capture (16kHz) + playback (24kHz)
- Camera capture (JPEG 768×768 at 1fps)
- Real-time transcription display

### Phase 4: Mobile PWA (Days 5-6)

- Responsive React, PWA manifest
- Voice-first UX, rear camera integration

### Phase 5: ESP32 Glasses (Day 6-7)

- ESP32 I2S mic/speaker over WiFi WebSocket
- Or: Python simulation script for demo

### Phase 6: Deployment & Demo (Day 7-8)

#### Cloud Deployment

- **Cloud Run**: Dockerfile, uvicorn, min_instances=1
- **Firestore**: sessions, personas, MCP configs
- **Terraform/gcloud scripts** → +0.2 bonus

#### Demo Video (≤ 4 min)

| Timestamp | Content |
|---|---|
| 0:00-0:30 | Problem: "AI assistants are single-device, siloed" |
| 0:30-1:30 | Voice conversation + persona switching + voice changes |
| 1:30-2:30 | MCP plugin toggle + E2B code execution in sandbox |
| 2:30-3:15 | Mobile camera: "What am I looking at?" live vision |
| 3:15-3:45 | Architecture diagram + cross-client action demo |
| 3:45-4:00 | Cloud Run console proof + Firestore data |

#### Bonus Points

- Blog post about build process → **+0.6**
- Automated deployment (Terraform) → **+0.2**
- GDG membership → **+0.2**

---

## 11. Tech Stack — Rapid Prototyping & Deployment

> **Philosophy**: Optimize for hackathon speed — fewest config files, fastest feedback loops, minimal boilerplate. Ship a working prototype, not a production system.

## Complete Stack

| Layer | Technology | Why This Choice |
|---|---|---|
| **Backend Runtime** | **Python 3.12+** | ADK is Python-first; best docs, samples, community support |
| **Package Manager** | **uv** | 10-100x faster than pip; replaces pip + venv + pip-tools in one binary; lockfile support; instant installs |
| **Web Framework** | **FastAPI** | Async native, WebSocket support, auto-generated OpenAPI docs, perfect for ADK `run_live()` |
| **ASGI Server** | **uvicorn** | Production-grade async server for FastAPI |
| **AI Framework** | **Google ADK** ( `google-adk` ) | Core hackathon requirement; `run_live()`, `McpToolset`, multi-agent, callbacks |
| **AI Model** | **Gemini 2.5 Flash Native Audio** | Live streaming, voice personas, affective dialog, proactive audio |
| **Frontend Framework** | **React 19 (JavaScript — no TypeScript)** | Fastest prototyping; skip type definitions overhead; user preference |
| **Frontend Build** | **Vite** | Sub-second HMR, minimal config, fastest dev server available |
| **Frontend Styling** | **Tailwind CSS** | Utility-first = rapid UI building without writing CSS files |
| **Component Library** | **shadcn/ui** (copy-paste, not npm dependency) | Beautiful pre-built components, fully customizable, no lock-in |
| **State Management** | **Zustand** | Minimal boilerplate (3 lines to create a store), no providers/context wrappers |
| **Database** | **Firestore** | Serverless, real-time listeners, GCP native, generous free tier, zero config |
| **File/Image Storage** | **Cloud Storage (GCS)** | Generated images, session media, user uploads |
| **Auth** | **Firebase Auth** | Google/email sign-in in minutes, GCP native, free tier |
| **Deployment** | **Cloud Run** | WebSocket support, auto-scaling, Docker-based, pay-per-use |
| **Container** | **Docker** | Cloud Run requirement; reproducible builds |
| **Sandbox** | **E2B** | Secure code execution, 100+ MCP gateway, <200ms cold start |
| **Realtime Comms** | **WebSocket (native)** | Bidi streaming for PCM audio + JSON events; no library needed |
| **Audio Processing** | **Web Audio API + AudioWorklet** | Browser-native audio capture (16kHz) + playback (24kHz) |

| Layer | Technology | Why This Choice |
|---|---|---|
| **Desktop Client** | **Python + pystray + pyautogui** | Lightweight system tray app, computer use agent |
| **Chrome Extension** | **Manifest V3 + vanilla JS** | Voice-activated browser control, minimal dependencies |
| **Mobile** | **React (responsive PWA)** | Same codebase as web dashboard, add to home screen |
| **Charts** | **Recharts** or **Chart.js** | GenUI dynamic chart rendering, React-friendly |
| **Icons** | **Lucide React** | Consistent icon set, tree-shakeable |
| **HTTP Client (frontend)** | **fetch (native)** | No need for axios; native is sufficient |
| **CI/CD** | **Cloud Build** or **GitHub Actions** | Auto-deploy on git push |
| **IaC (bonus)** | **Terraform** | Automated GCP infra provisioning → +0.2 bonus points |
| **Secrets** | **Google Secret Manager** | API keys, MCP credentials — never in env files for prod |
| **Monitoring** | **Cloud Logging + Cloud Trace** | GCP native, zero-config observability |
| **Linting** | **Ruff** (Python) + **ESLint** (JS) | Ruff is 10-100x faster than flake8/pylint; ESLint standard |
| **Formatting** | **Ruff format** (Python) + **Prettier** (JS) | Consistent code style, zero debates |

## Quick Start Commands

```
# === Backend Setup ===
pip install uv                              # Install uv (one-time)
uv init backend && cd backend               # Create project
uv add google-adk google-genai              # Core AI
uv add fastapi uvicorn python-dotenv        # Web server
uv add google-cloud-firestore               # Database
uv add e2b-code-interpreter                 # Sandbox
uv add websockets                           # WebSocket client utils
uv add ruff                                 # Linting + formatting

# === Frontend Setup ===
npm create vite@latest dashboard -- --template react   # React JS (not TS!)
cd dashboard
npm install
npm install -D tailwindcss @tailwindcss/vite           # Styling
npm install zustand recharts lucide-react               # State, charts, icons
npm install firebase                                   # Auth + Firestore client

# === Desktop Client Setup ===
cd .. && uv init desktop-client && cd desktop-client
uv add pystray Pillow pyautogui pyperclip pygetwindow psutil websockets

# === Chrome Extension ===
mkdir -p chrome-extension  # Manifest V3 + vanilla JS — no build step needed

# === Deployment ===
gcloud run deploy agent-hub \
  --source . \
  --region us-central1 \
  --allow-unauthenticated \
  --min-instances 1 \
  --session-affinity
```

## Why These Specific Choices

| Decision | Alternatives Considered | Why We Chose This |
|---|---|---|
| **uv** over pip/poetry | pip, poetry, pdm, conda | uv is 10-100x faster, handles venvs + lockfiles + installs in one tool |
| **JS** over TS | TypeScript | Hackathon speed — skip type definitions, compilation, tsconfig. Prototype first, refactor later |
| **Vite** over CRA/Next.js | Create React App, Next.js, Remix | Vite: zero-opinion, fastest HMR, no SSR complexity. We don't need Next.js routing/SSR |
| **Firestore** over PostgreSQL | Cloud SQL, Supabase, MongoDB Atlas | Firestore: serverless (no connection pools), real-time listeners for dashboard updates, generous free tier, GCP native |
| **Zustand** over Redux/Context | Redux Toolkit, React Context, Jotai | Zustand: 3-line store creation, no provider wrappers, works outside React components |
| **shadcn/ui** over MUI/Chakra | Material UI, Chakra UI, Ant Design | shadcn: copy-paste (not npm dependency), fully customizable, beautiful defaults, Tailwind-native |
| **Firebase Auth** over Auth0/Clerk | Auth0, Clerk, Supabase Auth | Firebase Auth: GCP native, Google sign-in in 10 lines, free tier |
| **Ruff** over flake8/pylint | flake8, pylint, black, isort | Ruff: replaces flake8+pylint+black+isort in one tool, 100x faster |
| **pystray** over Electron | Electron, Tauri, PyQt | pystray: 50 lines for a tray app vs 5000+ for Electron; we need a lightweight agent, not a GUI app |
| **Manifest V3 + vanilla JS** | Plasmo, CRXJS | No build step, no framework overhead; extension is thin — just WebSocket + chrome APIs |

### Project Structure

```
agent-hub/
├── backend/                    # Python + FastAPI + ADK
│   ├── pyproject.toml          # uv project config
│   ├── uv.lock                 # Lockfile
│   ├── main.py                 # FastAPI app, WebSocket endpoints
│   ├── agents/                 # ADK agent definitions
│   │   ├── personas.py         # Agent persona configs
│   │   └── router_agent.py     # Root agent with sub_agents
│   ├── services/               # Business logic
│   │   ├── client_registry.py  # Track connected clients
│   │   ├── mcp_manager.py      # Dynamic MCP loading
│   │   └── e2b_service.py      # E2B sandbox management
│   ├── tools/                  # ADK custom tool functions
│   │   ├── desktop_tools.py    # Computer use tools
│   │   ├── image_gen.py        # Nano Banana interleaved output
│   │   └── cross_client.py     # Cross-client action tools
│   ├── Dockerfile              # Cloud Run container
│   └── .env                    # Local dev secrets
├── dashboard/                  # React (JS) + Vite
│   ├── package.json
│   ├── vite.config.js
│   ├── src/
│   │   ├── App.jsx
│   │   ├── components/         # UI components
│   │   │   ├── Chat.jsx        # Voice chat + transcription
│   │   │   ├── GenUI.jsx       # Dynamic component renderer
│   │   │   ├── PersonaManager.jsx
│   │   │   ├── MCPStore.jsx
│   │   │   └── ClientStatus.jsx
│   │   ├── stores/             # Zustand stores
│   │   │   └── appStore.js
│   │   ├── hooks/              # Custom hooks
│   │   │   ├── useWebSocket.js
│   │   │   └── useAudio.js
│   │   └── lib/                # Utilities
│   │       └── firebase.js
│   └── public/
├── desktop-client/             # Python tray app
│   ├── pyproject.toml
│   ├── main.py                 # Tray app + WebSocket + automation
│   └── tools.py                # pyautogui actions
├── chrome-extension/           # Manifest V3
│   ├── manifest.json
│   ├── background.js           # Service worker + WebSocket
│   ├── content.js              # Page interaction
│   ├── popup.html + popup.js   # Extension popup UI
│   └── icons/
├── deploy/                     # IaC (bonus points)
│   ├── main.tf                 # Terraform config
│   └── deploy.sh               # gcloud deploy script
├── docs/                       # Documentation
│   └── architecture.png
└── README.md
```

---

## 12. Judging Criteria Alignment — Score Maximization Strategy

> Judges score 1–5 on three weighted criteria, then add up to +1.0 bonus.
> **Our target: 5.0 scored + 1.0 bonus = 6.0 / 6.0**
> This section maps every feature to the exact judging language and predicts where we'll score highest and where risk remains.

---

### 12.1 Innovation & Multimodal User Experience — 40% (Target: 5/5)

This is the **highest-weighted criterion** and our strongest differentiator. A perfect score here is the path to Grand Prize.

#### What Judges Are Looking For (Exact Rubric Language)

> *"Beyond Text" Factor: Breaks the text-box paradigm? Interaction is natural, immersive, superior to standard chat?*
> *Live Agents: Handles interruptions (barge-in) naturally? Distinct persona/voice?*
> *Fluidity: "Live" and context-aware, or disjointed and turn-based?*

### Feature-to-Rubric Mapping

| Rubric Phrase | Our Answer | Evidence in Demo | Risk Level |
|---|---|---|---|
| **"Breaks the text-box paradigm"** | 5 simultaneous client types (web dashboard, mobile PWA, Chrome extension, desktop tray app, ESP32 glasses) — user speaks from ANY device to the same agent brain | Voice conversation on web → switch to phone camera → note appears on dashboard. Zero text boxes in primary flow | **LOW** — multi-client is inherently beyond-text |
| **"Natural, immersive, superior to standard chat"** | Gemini native audio with affective dialog (agent senses user emotion and adapts tone), proactive audio (agent speaks first when context warrants), persistent persona voices | Agent notices user sounds frustrated, softens tone. Agent proactively says "I noticed you opened a financial doc — want me to analyze it?" | **MEDIUM** — affective dialog quality depends on model; proactive audio needs precise tuning to avoid being annoying |
| **"Handles interruptions naturally"** | Gemini Live API VAD + barge-in built-in. AudioStreamer stops playback + clears queue on interrupt event. Agent acknowledges interruption gracefully | User interrupts mid-sentence, agent stops immediately, says "Sure, go ahead." No awkward overlap or double-speaking | **LOW** — native Gemini capability, just needs clean audio pipeline |
| **"Distinct persona/voice"** | 5+ personas (e.g., Nova/Analyst/Charon, Atlas/Coder/Kore, Sage/Researcher/Aoede) each with unique voice, system instructions, MCP set. Users can create custom personas via dashboard or voice command | Switch from Nova (calm analyst) to Atlas (precise coder) — voice and behavior change instantly. "Create a new persona called Chef with a warm voice" → done in real-time | **LOW** — `speech_config` per persona is a config, not an engineering challenge |
| **"Live and context-aware"** | Single ADK session shared across all client types. Cross-client actions: "Save this to my dashboard" from glasses → appears on web. Session persists across disconnects via `SessionResumptionConfig` | User on mobile says "remember this recipe." Later on desktop: "what was that recipe?" — agent recalls it. Cross-client note-taking visible in real-time | **MEDIUM** — session sharing across clients needs careful session ID management |

| Rubric Phrase | Our Answer | Evidence in Demo | Risk Level |
|---|---|---|---|
| **"Not disjointed / turn-based"** | BIDI streaming mode (simultaneous speech). Binary WebSocket frames for audio (no JSON overhead → lower latency). AudioWorklet on separate thread (no main-thread jank). Pre-loaded MCP connections | Continuous natural conversation with no perceivable round-trip delay. Agent can be speaking while user starts a new thought. Tool calls happen with "let me check..." verbal feedback, no dead silence | **MEDIUM** — perception of fluidity depends on network latency + tool call speed |

### What Makes Us Score 5 Instead of 4

Most entries will be voice chatbots on a single web page. To score 5, judges need to see something they've **never seen before**:

1. **Cross-client actions** — No other entry will have "tell my glasses to highlight this, save the screenshot to my web dashboard." This is the single biggest differentiator.
2. **GenUI** — Agent doesn't just speak; it renders live charts, tables, code blocks, timelines on the dashboard while talking. Judges see audio + visual simultaneously.
3. **MCP plugin store** — User adds capabilities mid-conversation like installing an app. "Enable Brave Search" → agent immediately performs a web search. No restart.
4. **Voice-created personas** — Dynamic at runtime, not just pre-configured. Shows the agent is truly intelligent about its own configuration.

### Where We Could Lose a Point

| Risk | Impact | Mitigation |
|---|---|---|
| Tool call latency causes awkward 2-3s silence | Feels "disjointed" (kills fluidity score) | Agent gives verbal acknowledgment: "Let me look that up..." + show loading spinner on GenUI panel |
| Proactive audio triggers at wrong time | Annoying, not "natural" | Conservative triggers: only when user hasn't spoken for 30s AND new relevant context arrives |
| ESP32 glasses feel like a gimmick | Judges roll eyes at unfinished hardware | Frame as "architecture extensibility proof" with WebSocket protocol diagram. Show the Python simulator as stand-in. Don't oversell. |
| GenUI components look janky | Undermines "immersive" claim | Use shadcn/ui for every component. Dark mode polished. Recharts with smooth animations. Test every component type before demo. |

---

## 12.2 Technical Implementation & Agent Architecture — 30% (Target: 5/5)

### What Judges Are Looking For (Exact Rubric Language)

> *"Google Cloud Native: Effective use of Google GenAI SDK/ADK? Robust hosting (Cloud Run, Vertex AI, Firestore)?"*
> *"System Design: Sound agent logic? Handles errors, API timeouts, edge cases gracefully?"*
> *"Robustness: Avoids hallucinations? Evidence of grounding?"*

### Feature-to-Rubric Mapping

| Rubric Phrase | Our Answer | Evidence | Risk |
|---|---|---|---|
| **"Effective use of ADK"** | `run_live()` for bidi streaming, `McpToolset` for plugin system, multi-agent with `sub_agents` (persona delegation), `RunConfig` with streaming_mode=BIDI + speech_config + proactivity + affective_dialog, `SessionService` for persistence, Callbacks for guardrails, Context Compression for unlimited sessions | Code shows: root_agent → persona sub-agents via `transfer_to_agent`, live_request_queue sending audio/text/images, McpToolset wrapping 10+ MCP servers | **LOW** — we use nearly every ADK feature |
| **"Robust hosting"** | Cloud Run (min_instances=1, WebSocket support, autoscaling), Firestore (sessions + personas + MCP configs + chat history), Firebase Auth (Google sign-in), GCS (generated images/files) | Cloud Run console showing service, Firestore collections in Firebase console, `gcloud run deploy` in Terraform/scripts | **LOW** — standard GCP stack |
| **"Sound agent logic"** | Root LlmAgent routes to persona sub-agents (each has unique instruction + tools + voice). TaskArchitect (CustomAgent) decomposes complex tasks into sub-agent pipelines. Cross-client tools call device-specific APIs. E2B sandbox isolates code execution | Architecture diagram showing agent hierarchy. Code showing `sub_agents=[nova, atlas, sage, spark, claire]` with tool delegation | **LOW** — ADK multi-agent pattern is well-documented |
| **"Handles errors, timeouts, edge cases"** | ADK Callbacks: `before_model_callback` (prompt injection guard), `after_model_callback` (hallucination check), `on_tool_error_callback` (MCP failure fallback). WebSocket reconnection with exponential backoff. E2B sandbox timeout handler. Rate limiting. Circuit breaker for MCP connections | Error handling code in callbacks. Demo: disable an MCP mid-conversation → agent says "That tool is unavailable, let me try another approach" | **MEDIUM** — needs explicit implementation and testing |
| **"Avoids hallucinations / grounding"** | Multi-layer grounding: (1) `google_search` tool (built-in), (2) Brave Search MCP (web), (3) Wikipedia MCP (facts), (4) Context7 MCP (code docs), (5) E2B execution verifies code. Anti-hallucination system instructions in every persona. `after_model_callback` checks for confidence markers | Ask agent a factual question → see it call google_search → cite the source. Ask for code → E2B runs it → confirms output before presenting | **MEDIUM** — grounding only works if agent actually uses tools; depends on prompt engineering |

### ADK Feature Depth Score

Judges evaluating "effective ADK usage" will mentally check off features. Here's our coverage:

| ADK Feature | Used? | How |
|---|---|---|
| `LlmAgent` | ✅ | Root agent + 5 persona sub-agents |
| `CustomAgent` (BaseAgent) | ✅ | TaskArchitect meta-orchestrator |
| `SequentialAgent` | ✅ | TaskArchitect sub-pipelines |
| `ParallelAgent` | ✅ | TaskArchitect concurrent tool calls |
| `run_live()` | ✅ | Bidi audio streaming — core of the product |
| `LiveRequestQueue` | ✅ | Audio/text/image ingestion to agent |
| `RunConfig` | ✅ | streaming_mode, speech_config, response_modalities, proactivity, affective_dialog |
| `McpToolset` | ✅ | Dynamic MCP plugin system |
| `SessionService` | ✅ | InMemory (dev) → Firestore-backed (prod) |
| `Callbacks / Plugins` | ✅ | Error handling, safety guardrails, logging |
| `Artifacts` | ✅ | Save generated code, images, documents to GCS |
| `Context Compression` | ✅ | Unlimited session length |
| `google_search` tool | ✅ | Built-in grounding |
| `transfer_to_agent` | ✅ | Persona switching via sub-agent delegation |
| `SessionResumptionConfig` | ✅ | Reconnection without losing context |

**14 out of 15 major ADK features used.** Only A2A Protocol skipped (overkill for hackathon).

### Google Cloud Services Used

| Service | Purpose | Judges See |
|---|---|---|
| **Cloud Run** | Backend hosting (FastAPI + ADK) | Cloud Run console in demo, autoscaling |
| **Firestore** | Sessions, personas, MCP configs, chat history | Firebase console showing collections |
| **Firebase Auth** | Google sign-in authentication | Login page → Google OAuth flow |
| **Cloud Storage (GCS)** | Generated images, code artifacts, file storage | Images appearing on dashboard from agent |
| **Cloud Logging** | Request/error logging | Log Explorer showing real requests |
| **Artifact Registry** | Docker container storage | Docker push → Cloud Run deploy |
| **Secret Manager** | MCP API keys, service account keys | Secure config, not hardcoded |

**7 Google Cloud services.** Judges evaluating "Cloud Native" will see deep GCP usage, not just "I deployed to Cloud Run."

---

## 12.3 Demo & Presentation — 30% (Target: 5/5)

### What Judges Are Looking For (Exact Rubric Language)

> *"The Story: Video clearly defines problem and solution?"*
> *"The Proof: Architecture diagram clear? Visual proof of Cloud deployment?"*
> *"The 'Live' Factor: Video shows actual working software?"*

### Demo Video Script (4 Minutes)

| Time | Segment | What's Shown | Judges Feel |
|---|---|---|---|
| **0:00–0:30** | **Hook & Problem** | "AI assistants live in text boxes on single screens. You can't speak to your AI while wearing safety glasses on a factory floor. You can't add new capabilities without waiting for the next software update. You can't switch devices mid-thought." Quick montage of ChatGPT text box, single-screen limitation | "They understand the problem. This is relatable." |
| **0:30–1:00** | **Solution Reveal** | "Agent Hub — one AI brain, every device." Show architecture diagram (professional, animated). Highlight: multi-client, MCP store, agent personas, GenUI, E2B sandbox. Quick animation showing glasses → phone → dashboard all connected | "Clear architecture. They've thought this through." |
| **1:00–2:15** | **Live Demo: Voice + GenUI** | Open web dashboard. Have a natural voice conversation with the agent. Ask: "What's Tesla's stock performance this year?" Agent speaks response AND renders a live Recharts line graph on the dashboard simultaneously. Ask: "Compare with Ford" → GenUI table appears inline. Switch persona from Nova (analyst) to Atlas (coder): "Write a Python function to calculate compound interest." Agent speaks the explanation while code block renders with syntax highlighting. Click "Execute" → E2B runs it → output appears | "This is REAL. Voice + visuals together. Persona switch was seamless. Code actually ran." |
| **2:15–2:45** | **Live Demo: MCP Plugin Store** | Open Plugin Store page. Enable "Brave Search" with one toggle. Return to chat: "Search for the latest news about Gemini API." Agent calls Brave Search, results appear. Disable it. Try again → agent says "Web search isn't available. I can use my training knowledge instead." | "Dynamic capabilities. Error handling is graceful." |
| **2:45–3:15** | **Live Demo: Cross-Client + Mobile Camera** | Open mobile PWA on phone (or responsive view). Point camera at an object (a book, product, etc.). Agent sees it, describes it, says "I've saved this analysis to your dashboard." Switch to desktop dashboard → the analysis card is there | "Cross-device. The killer feature. This is novel." |
| **3:15–3:35** | **Cloud Proof** | Quick screen recording: Cloud Run console (service running, 200 requests), Firestore (sessions collection with real data), Cloud Logging (request logs), `terraform apply` output | "Properly deployed. Not localhost only." |
| **3:35–4:00** | **Close & Impact** | "Agent Hub proves AI agents don't belong in text boxes. Any device — glasses, phone, browser, desktop — connects to one intelligent hub. New capabilities install in one click. This is how humans should interact with AI." Show architecture one more time with all clients connected | "They nailed the vision. This is the winner." |

### Demo Risk Mitigation

| Risk | Probability | Mitigation |
|---|---|---|
| API quota exhaustion | Medium | Pre-test 30min before recording. Have backup recording from previous session |
| Network latency causes delay | Medium | Record on fast WiFi. Use Cloud Run in nearest region. If delay occurs, narrate: "The agent is processing..." |
| Audio quality issues | Low | Use a quality USB microphone. Test audio levels before recording. Browser in quiet room |
| MCP server down | Low | Brave Search MCP is stable. Have a backup: show Google Search tool instead |
| E2B sandbox fails | Low | Pre-warm a sandbox. If it fails in recording, use the backup take |
| Voice sounds robotic | Low | Use `gemini-2.5-flash-native-audio` which has natural voices. Test different voices, pick the best |

**Rule: Record 3 takes. Use the best one. Add post-production narration overlays if any segment has audio issues.**

---

## 12.4 Bonus Points — +1.0 (Target: 1.0/1.0)

| Bonus | Points | Our Plan | Effort | Deadline |
|---|---|---|---|---|
| **Published Content** | +0.6 | Blog post on dev.to or Medium: "How I Built a Multi-Device AI Agent Hub with Gemini Live API & Google ADK." Include: problem statement, architecture diagram, code snippets of AudioWorklet pipeline, GenUI pattern, MCP plugin store, lessons learned. Share on X/LinkedIn with #GeminiLiveAgentChallenge | 3–4 hours writing | March 15 (2 days before deadline) |
| **Automated Cloud Deployment** | +0.2 | Terraform files in `deploy/terraform/`: `main.tf` (Cloud Run + Firestore + GCS + Secret Manager), `variables.tf`, `outputs.tf`. Plus `deploy/scripts/deploy.sh` for one-command deploy | 2–3 hours | March 14 |
| **GDG Membership** | +0.2 | Join any Google Developer Group on developers.google.com. Link profile URL in Devpost submission | 10 minutes | Do NOW |

**Total bonus: +1.0. Combined with 5.0 scored = 6.0/6.0 maximum.**

---

### 12.5 Prize Targeting Strategy

We're eligible for multiple prize categories. Strategy: optimize for Grand Prize, naturally qualify for subcategory prizes.

| Prize | Cash | Our Alignment | Win Probability |
|---|---|---|---|
| **Grand Prize** (best across all) | $25,000 + Cloud Next tickets + demo opportunity | We maximize ALL three criteria + bonus. Multi-client differentiation is unique across categories | **HIGH** — if execution is polished |
| **Best Live Agent** | $10,000 | Our category. Voice-first, barge-in, multi-client, personas, proactive audio — checks every Live Agent box | **HIGH** — this is our primary target |
| **Best Multimodal Integration & UX** | $5,000 | Voice + vision + GenUI + cross-device. Literally the definition of multimodal | **HIGH** — secondary target |
| **Best Technical Execution** | $5,000 | 14 ADK features, 7 GCP services, MCP ecosystem, E2B sandbox, binary WS audio | **MEDIUM-HIGH** — depends on competitor depth |
| **Best Innovation & Thought Leadership** | $5,000 | Multi-client hub concept is novel. MCP plugin store extensibility. Blog post with deep technical content | **MEDIUM** — "innovation" is subjective |

> Per rules: if we win both Best Live Agent + a subcategory prize, the subcategory goes to next project. But Grand Prize stacks with one category prize.

---

## 13. Judging Criteria Critique — Thinking Like The Judge

> This section adopts the judges' mindset. For each criterion, we ask: "If I were a judge reviewing 200+ entries in 2 weeks, what would make me give a 5 vs a 3? What red flags would I watch for? Where does Agent Hub have gaps?"

---

### 13.1 Judge's Perspective: Innovation & Multimodal UX (40%)

#### How Judges Actually Evaluate This

A judge reviewing the "Live Agent" category has seen dozens of voice chatbots by entry #50. They're fatigued. A 3/5 entry is "another voice chat with a Gemini backend." A 4/5 has one clever twist — maybe it adds camera, or has a nice UI. A **5/5 makes the judge lean forward and say "I haven't seen this before."**

#### What earns a 5:

- Something architecturally novel (not just "I used a different API")
- Multi-modal means MULTIPLE modalities working TOGETHER, not sequentially
- The user experience is genuinely better than existing products
- The demo shows a capability no other tool offers

**What our entry does that NO other entry will have:**

1. **Cross-client action orchestration** — A command on one device triggers visible results on another device in real-time. Not "sync" — active orchestration by the AI agent.
2. **Runtime-extensible agent via MCP store** — Users install capabilities like apps. Agent's skill set changes during the conversation. No other entry will have a visual plugin store.
3. **GenUI** — Agent renders structured UI components (charts, tables, code blocks) inline in chat while simultaneously speaking audio. This is audio+visual multimodality in one message.

**Red flags judges would notice:**

- **Half-baked multi-client**: If ESP32 or mobile feels like an afterthought, it undermines the "hub" claim. Better to show 2 polished clients than 5 broken ones.
- **Fake fluidity**: If there's a perceivable pause every time the agent calls a tool, it's "disjointed" — the exact opposite of what the rubric wants.
- **Gimmicky personas**: If persona switching is just "different system prompt + same voice," judges won't be impressed. Each persona needs a distinct voice AND visibly different behavior.

**Gap Analysis**

| Gap | Severity | Fix |
|---|---|---|
| Tool call latency breaks perceived fluidity | **HIGH** | Agent must give verbal acknowledgment ("Let me check...") + GenUI shows a loading skeleton during tool execution. Never have dead silence. |
| ESP32 without real hardware feels like slide-ware | **MEDIUM** | Show Python simulator sending real WebSocket messages. Frame it as "protocol-ready architecture" not "we built smart glasses." 10% of demo time max. |
| Proactive audio could annoy rather than impress | **MEDIUM** | Conservative triggers with 30s quiet threshold. Demo a scripted scenario where proactive behavior is obviously helpful (e.g., agent notices file opened on desktop, offers to analyze). |
| GenUI components might not render properly under live conditions | **MEDIUM** | Pre-test every GenUI type (chart, table, code, image, timeline). Use real data. Have 2-3 reliable query → GenUI response pairs scripted for demo. |

---

## 13.2 Judge's Perspective: Technical Implementation (30%)

**How Judges Actually Evaluate This**

Technical judges will **skim the code**, not read every file. They look for:

1. Is the architecture diagram clean and honest? (Does the code match the diagram?)
2. Are they using ADK properly, or just calling `generate_content()` in a loop?
3. Did they handle the hard problems (errors, state, scaling) or just the happy path?
4. Is this deployable, or only works on localhost?

**What earns a 5:**

- Architecture diagram matches actual code (no "planned" boxes)
- Multiple ADK agent types used purposefully (not just LlmAgent)
- Error handling is visible in the code (not just documented)
- Cloud deployment is real (Cloud Run URL works during judging period)
- Grounding is implemented, not just mentioned in README

**What our entry does well:**

- 14 ADK features actively used (more than any guide or tutorial shows)
- Multi-agent hierarchy with genuine delegation logic (root → personas → TaskArchitect)
- WebSocket + AudioWorklet audio pipeline handles the hardest real-time engineering problem
- MCP integration is dynamic (runtime enable/disable) not hardcoded

**Red flags judges would notice:**

- **InMemorySessionService in production** — Judges know Cloud Run is stateless. If sessions don't survive a restart, the architecture is broken.
- **No error handling shown** — If the demo only shows happy path, judges assume there IS no error handling.
- **"Uses Firestore" but only for one thing** — If Firestore only stores user profiles, that's not "effective use." We should show sessions + personas + MCP configs + chat history + artifacts.
- **Hardcoded API keys** — Instant credibility loss. Must use Secret Manager or environment variables.

### Gap Analysis

| Gap | Severity | Fix |
|---|---|---|
| Session persistence across Cloud Run restarts | **CRITICAL** | Implement Firestore-backed SessionService. Test: deploy → chat → redeploy → session resumes |
| No error handling demo | **HIGH** | Intentionally show error recovery in demo: disable an MCP → agent handles it gracefully. This is a 30-second "wow" for technical judges. |
| No callback/plugin implementation shown | **HIGH** | Implement: `before_model_callback` (input sanitization), `after_model_callback` (hallucination check), `on_tool_error_callback` (MCP failure). Even barebones implementations show awareness. |
| Hallucination prevention is passive | **MEDIUM** | Make it active and demonstrable: agent cites sources in responses. System instructions say "Always use search tools for factual claims. Cite the source URL." Show this in demo. |
| Context window overflow for long sessions | **MEDIUM** | Enable `context_window_compression=True` in RunConfig. One line of code. Mention in blog post as "production consideration." |
| Cloud Logging not configured | **LOW** | Add structured logging with `google-cloud-logging`. Show log explorer in demo. Takes 30 minutes to set up, big impression on judges. |

---

## 13.3 Judge's Perspective: Demo & Presentation (30%)

### How Judges Actually Evaluate This

Judges watch the video at 1x or 1.5x speed. They decide within the first 30 seconds whether to pay full attention or zone out. Their rubric has three questions:

1. **"Do I understand what this solves?"** (The Story) — If the problem isn't clear by 0:30, they're already scoring 3/5 mentally.
2. **"Is this actually working?"** (The Live Factor) — They're specifically looking for actual software, not slides or mockups. Real data, real API calls, real audio.
3. **"Could this actually be deployed?"** (The Proof) — Architecture diagram + Cloud console screenshot. If missing, they assume localhost-only.

**What earns a 5:**

- Opens with a relatable problem, not "Hi, I'm ___, and I built..."
- Shows working software within the first 60 seconds
- Has at least one "holy crap" moment where judges pause and rewind
- Architecture diagram is clean, professional, and matches what was shown
- Cloud deployment is proven with visual evidence (console screenshots, Firestore data)

**What kills the score:**

- Spending 2 minutes explaining what they're GOING to show before showing it
- Lag, errors, or crashes during the demo
- Hard-to-hear audio or screen recordings too small to read
- No architecture diagram
- Running on localhost only

**Our "Holy Crap" Moments (Planned)**

1. **Cross-client action** (2:45–3:15): Point phone camera at something → agent describes it → "I've saved this to your dashboard" → switch to desktop → it's there. This is the rewind moment.
2. **GenUI during voice** (1:00–1:30): Ask about stock performance → agent speaks the answer while a chart renders in real-time on the dashboard. Audio + visual simultaneously.
3. **Persona live switch** (1:45–2:00): Mid-conversation "Switch to Atlas" → voice changes instantly → asks for code → code block renders → "Execute it" → E2B runs → output appears.
4. **MCP toggle** (2:15–2:45): Enable a plugin on the fly → use it immediately → disable it → agent gracefully falls back. Runtime extensibility.

**Gap Analysis**

| Gap | Severity | Fix |
|---|---|---|
| 4 minutes is extremely tight for all features | **HIGH** | Prioritize ruthlessly. Cut anything that doesn't directly score points. The 4 "holy crap" moments above are the core. Everything else is supporting context. |
| Audio quality in screen recording | **HIGH** | Use OBS Studio with separate audio tracks. Record mic with a USB condenser. Post-process audio in Audacity if needed. |
| Architecture diagram quality | **MEDIUM** | Use Excalidraw or Figma (not ASCII art). Include: all client types, WebSocket connections, Cloud Run, ADK agent tree, Firestore, GCS, MCP servers, E2B. Color-code by layer. |
| Cloud proof feels rushed | **MEDIUM** | Dedicate 20s to a clean screen recording: Cloud Run console → Service running → Firestore → Sessions collection → Cloud Logging → Real request logs. Pre-record this segment separately. |

---

### 13.4 Comparative Weakness: What Competitors Are Likely Doing Better

| Competitor Type | Their Advantage | Our Counter |
|---|---|---|
| **Solo deep-tech entries** | One feature polished to perfection (e.g., perfect real-time translator with amazing latency) | We counter with breadth + architecture novelty. No solo entry will have multi-client + MCP store + GenUI. |
| **Team entries with design resources** | Beautiful UI, smooth animations, polished branding | We counter with shadcn/ui + Tailwind (professional by default). Dark mode polished. GenUI components give visual richness without custom design. |
| **Entries with real hardware** | Physical ESP32 glasses working on camera | We counter by making hardware an “extensibility proof” not the main pitch. Our main pitch is the hub architecture. |
| **Enterprise-focused entries** | Deep Vertex AI integration, RAG pipelines, evaluation frameworks | We counter with broader Google Cloud usage (7 services) + live streaming (harder engineering than RAG). |
| **Creative Storyteller entries** | Gorgeous interleaved media output | Different category. Not competing directly. |

---

## 14. Enhancement Recommendations — Ordered by Judge Impact

> Numbered by priority. Each enhancement is tagged with which scoring criterion it boosts and the estimated implementation time.

## TIER 1 — Must Have (Score Differentiators)

| # | Enhancement | Criterion | Impact | Time |
|---|---|---|---|---|
| 1 | **Verbal tool-call acknowledgment** — Agent says "Let me check..." or "One moment..." while tools execute. Never dead silence. | Innovation 40% (fluidity) | Prevents 1-point deduction on "disjointed" | 1 hour |
| 2 | **GenUI loading skeletons** — Show shimmer/skeleton in chat while agent generates structured response. Transitions to real content smoothly. | Innovation 40% (fluidity) | Feels polished, not broken | 2 hours |
| 3 | **Error recovery demo** — Intentionally disable an MCP in the demo and show the agent recover gracefully. | Technical 30% (error handling) | Judges specifically look for this. Most entries skip it. | 1 hour |
| 4 | **Firestore-backed session persistence** — Sessions survive Cloud Run restart. | Technical 30% (robustness) | Required for legitimate "production-ready" claim | 4 hours |
| 5 | **Active grounding with source citation** — Agent cites URLs/sources when answering factual questions. System instruction: "Always verify facts with search tools. Cite sources in responses." | Technical 30% (hallucination) | Makes grounding visible and verifiable to judges | 2 hours |
| 6 | **Architecture diagram — professional quality** — Excalidraw or Figma. Color-coded by layer. All components labeled. Exported as PNG for submission + shown in video. | Demo 30% (proof) | Missing diagram = auto lose 1 point | 3 hours |
| 7 | **Cross-client demo scenario** — Scripted end-to-end: phone camera → agent describes → saves to dashboard → switch to desktop → it's there. Rehearsed and tested. | Innovation 40% + Demo 30% | THE "wow" moment. This is the rewind-worthy clip. | 2 hours rehearsal |
| 8 | **Blog post** | Bonus +0.6 | 60% of bonus points | 3-4 hours |

## TIER 2 — Should Have (Polish & Depth)

| # | Enhancement | Criterion | Impact | Time |
|---|---|---|---|---|
| 9 | **ADK Callbacks** — `before_model_callback` (input sanitization), `after_model_callback` (response validation), `on_tool_error_callback` (MCP failure handling) | Technical 30% | Shows production mindset. Code is visible to judges who read the repo | 3 hours |
| 10 | **Context Compression enabled** — `context_window_compression=True` in RunConfig | Technical 30% | One line of code. Enables unlimited sessions. Easy win | 5 minutes |
| 11 | **Cloud Logging integration** — Structured logging to Cloud Logging. Show Log Explorer in demo proof segment. | Technical 30% | Judges see real prod telemetry, not just "it runs" | 1 hour |
| 12 | **Terraform deployment** — `deploy/terraform/main.tf` for Cloud Run + Firestore + GCS + Secret Manager | Bonus +0.2 | 20% of bonus. Automates reproducibility | 2-3 hours |
| 13 | **GDG membership** — Join Google Developer Group, link in submission | Bonus +0.2 | Free points. Do immediately | 10 minutes |
| 14 | **Voice-created personas** — "Create a new persona called Chef with a warm voice and cooking knowledge" → agent creates config in real-time | Innovation 40% | Shows the agent can configure itself. Judges haven't seen this. | 3 hours |
| 15 | **Artifact saving to GCS** — Generated code, analysis results, images stored as ADK Artifacts in Cloud Storage | Technical 30% | Shows ADK Artifacts feature usage (rare in entries) | 2 hours |

### TIER 3 — Nice to Have (If Time Permits)

| # | Enhancement | Criterion | Impact | Time |
|---|---|---|---|---|
| 16 | **Ambient/passive listening mode** — Agent listens silently and only speaks when something relevant is detected via proactive audio | Innovation 40% | Demonstrates cutting-edge Gemini native audio capability | 4 hours |
| 17 | **TaskArchitect DAG visualization** — GenUI renders a visual task breakdown timeline when agent decomposes a complex request | Innovation 40% | Beautiful visual + shows CustomAgent depth | 6 hours |
| 18 | **Chrome extension voice commands** — "Read this page" / "Fill this form" from voice | Innovation 40% | Another client type, impressive breadth | 8 hours |
| 19 | **Desktop computer-use agent** — Python tray app with screenshot → agent → click/type actions | Innovation 40% | Bonus client, shows extensibility | 8 hours |
| 20 | **ADK Evaluation tests** — Pytest-based evaluation of agent responses for accuracy | Technical 30% | Shows quality assurance mindset | 4 hours |

### Implementation Priority Matrix

```
                    HIGH IMPACT
                        |
      ┌─────────────────┼─────────────────┐
      │   TIER 1        │   TIER 2        │
      │  (1) Verbal ack │  (9) Callbacks  │
      │  (3) Error demo │  (14) Voice     │
      │  (5) Grounding  │       personas  │
LOW   │  (7) Cross-client │ (15) Artifacts │  HIGH
TIME  │       demo      │                 │  TIME
      │  (8) Blog post  │                 │
      ├─────────────────┼─────────────────┤
      │  (2) GenUI skel │  TIER 3         │
      │  (10) Context   │  (16) Ambient   │
      │       compress  │  (17) TaskArch  │
      │  (13) GDG       │       DAG       │
      │  (6) Arch diagram │ (18) Chrome ext │
      │  (11) Logging   │  (19) Desktop   │
      │  (12) Terraform │  (20) Evaluation │
      │  (4) Firestore  │                 │
      │      sessions   │                 │
      └─────────────────┼─────────────────┘
                    LOW IMPACT
```

**Execution order**: Start top-left (high impact, low time) → top-right → bottom-left → bottom-right.

---

## 15. Scope & Decisions

### MVP Definition — What Must Ship

Everything below MUST be in the final submission. If any item is missing, we risk failing Stage 1 (pass/fail) or losing significant points.

| # | Feature | Criterion Served | Pass/Fail? |
|---|---|---|---|
| 1 | FastAPI backend with ADK `run_live()` bidi streaming | Technical 30% | Pass/Fail |
| 2 | Web dashboard (React JS + Vite + Tailwind + shadcn/ui) — voice chat + GenUI + persona panel + MCP store | Innovation 40% | Pass/Fail |
| 3 | Mobile PWA — responsive dashboard + camera vision | Innovation 40% | Scoring |
| 4 | Agent personas with distinct voices (5 default: Nova, Atlas, Sage, Spark, Claire) | Innovation 40% | Scoring |
| 5 | MCP plugin store — visual UI, enable/disable plugins mid-session, 10+ MCPs | Innovation 40% + Technical 30% | Scoring |
| 6 | E2B sandboxed code execution — write code → execute → return output | Technical 30% | Scoring |
| 7 | Cross-client actions — action on one device triggers result on another | Innovation 40% | Key differentiator |
| 8 | GenUI — agent renders charts, tables, code blocks, cards inline in chat | Innovation 40% | Key differentiator |
| 9 | Google Search grounding + anti-hallucination instructions | Technical 30% | Scoring |
| 10 | Firebase Auth (Google sign-in) | Technical 30% | Required for multi-user |
| 11 | Firestore persistence — sessions, personas, MCP configs, chat history | Technical 30% | Pass/Fail viability |
| 12 | Cloud Run deployment — accessible via public URL during judging | Technical 30% + Demo 30% | Pass/Fail |
| 13 | ADK Callbacks — error handling, input sanitization, tool failure recovery | Technical 30% | Scoring |
| 14 | Context Compression — unlimited session length | Technical 30% | Scoring |
| 15 | Demo video ≤ 4 min — follows scripted structure from Section 12.3 | Demo 30% | Pass/Fail |
| 16 | Architecture diagram — professional quality, all components labeled | Demo 30% | Pass/Fail |
| 17 | README with setup instructions — clone → configure → deploy | Demo 30% | Pass/Fail |
| 18 | Blog post — dev.to or Medium, #GeminiLiveAgentChallenge | Bonus +0.6 | Scoring |
| 19 | Terraform/scripts — automated Cloud deployment | Bonus +0.2 | Scoring |
| 20 | GDG membership linked | Bonus +0.2 | Scoring |

## Stretch Goals (If Time Permits After MVP)

Ordered by judge-impact per hour of effort:

| Priority | Feature | Impact | Effort |
|---|---|---|---|
| S1 | Voice-created personas ("Create a Chef persona with a warm voice") | Innovation 40% — unique capability | 3 hours |
| S2 | Chrome extension — voice-activated browser tasks | Innovation 40% — another client type | 8 hours |
| S3 | Desktop tray app — computer use agent (screenshot → action) | Innovation 40% — another client type | 8 hours |
| S4 | TaskArchitect DAG visualization — visual task breakdown GenUI component | Innovation 40% + Technical 30% | 6 hours |
| S5 | Ambient/passive listening mode — proactive audio triggers | Innovation 40% — cutting-edge demo | 4 hours |
| S6 | Interleaved image generation via Nano Banana tool | Technical 30% — shows GenAI SDK depth | 4 hours |
| S7 | ESP32 glasses with real hardware | Innovation 40% — hardware demo | 12+ hours |
| S8 | ADK Evaluation framework — automated response quality testing | Technical 30% — production mindset | 4 hours |

## Out of Scope (Not for This Hackathon)

| Feature | Why Not |
|---|---|
| A2A protocol integration | Overkill for hackathon; no other agent to talk to |
| Custom voice training / TTS | Gemini native voices are sufficient; custom training requires separate infra |
| Multi-language UI | English-only for demo; i18n is trivial but time-consuming |
| Payment/billing for MCP services | Enterprise feature; not relevant to judging |
| Cron jobs / scheduled tasks | Background automation is cool but not "live" interaction |
| Multi-user collaboration | Single-user hub for MVP; multi-user is an architecture extension |
| Rate-limited public API | Not needed; only our clients connect to backend |

## Key Technical Decisions with Judging Rationale

| Decision | Choice | Why (Judges' Lens) |
|---|---|---|
| **Category** | Live Agents | Strongest fit for multi-client voice architecture. Least crowded with high-quality entries (most people will do simple voice chat). |
| **Backend** | Python 3.12+ / FastAPI | ADK is Python-first. FastAPI supports WebSockets natively. Judges see idiomatic ADK code. |
| **Package Manager** | uv | Fast, reliable, deterministic. `uv run` in README is cleaner than pip instructions. |
| **Frontend** | React 19 (JavaScript) + Vite | JS over TS for prototype speed. Judges don't grade TypeScript usage. Vite for fast iteration. |
| **Styling** | Tailwind CSS + shadcn/ui | Professional-looking UI with minimal design effort. Dark mode built-in. Judges see a polished product. |
| **State** | Zustand | Minimal boilerplate. No context provider nesting. Clean code for judges to read. |
| **Database** | Firestore | Serverless + real-time + GCP native. Judges see "Google Cloud" usage. Free tier for hackathon. |
| **Auth** | Firebase Auth | Google sign-in in 10 lines. GCP native. Shows security awareness. |
| **Audio Transport** | Binary WebSocket frames | 33% smaller than base64-in-JSON. Lower latency. Technically impressive in code review. |
| **Session (dev)** | InMemorySessionService | Zero setup for local development |
| **Session (prod)** | Firestore-backed custom service | Survives Cloud Run restarts. Judges test the deployed URL. |
| **Audio Model** | `gemini-2.5-flash-native-audio-preview` | Proactive audio + affective dialog. Most advanced voice model. Shows latest Gemini capability. |
| **Code Execution** | E2B Sandbox | Secure isolation + 100+ built-in MCPs. Impressive capability for judges. |
| **MCP Connection** | Stdio + StreamableHTTP | Cover both local and remote MCP servers. Shows MCP expertise. |
| **Audio Pipeline** | AudioWorklet (not ScriptProcessorNode) | Runs on separate thread, zero main-thread jank. Technically correct choice that judges who know Web Audio will appreciate. |
| **GenUI** | Agent returns structured JSON → React renders components | Novel pattern. Judges haven't seen this in other entries. |

## 16. Verification Checklist

### Stage 1 — Pass/Fail Requirements (MUST ALL PASS)

Failing ANY item here means automatic disqualification regardless of quality.

- [ ] **Category selected**: "Live Agents — Real-time Interaction (Audio/Vision)" on Devpost
- [ ] **Text description**: Devpost description includes: features, functionality, technologies used, data ources, findings & learnings
- [ ] **Public code repository**: GitHub URL is public and accessible
- [ ] **README with spin-up instructions**: Clone → install → env setup → run locally AND deploy to Cloud un. Tested by someone other than the author
- [ ] **Google Cloud deployment proof**: Screen recording of Cloud Run console showing service running · Firestore with real data. OR link to deployment code in repo
- [ ] **Architecture diagram**: Clear visual showing Gemini ↔ backend ↔ database ↔ frontend. Uploaded o Devpost image carousel or file upload
- [ ] **Demo video ≤ 4 min**: Uploaded to YouTube or Vimeo. Publicly visible (not unlisted-only if Devpost equires public). Shows ACTUAL working software. English audio or English subtitles
- [ ] **Uses Gemini model**: `gemini-2.5-flash-native-audio-preview` confirmed in code
- [ ] **Uses Google ADK**: `google-adk` in dependencies, `Agent`, `Runner`, `run_live()` in code
- [ ] **Uses Google Cloud**: Cloud Run deployment confirmed, Firestore reads/writes confirmed
- [ ] **Complies with Google Cloud AUP**: No prohibited content, no abuse of APIs
- [ ] **Created during contest period**: Git history confirms all code is new

### Stage 2 — Innovation & Multimodal UX (40%) Verification

*"If I were a judge, would I give this a 5/5?"*

- [ ] **Voice conversation works end-to-end** — speak into browser, hear agent respond with correct ontent. No crashes, no long silences
- [ ] **Barge-in works** — interrupt agent mid-response, agent stops speaking immediately and cknowledges
- [ ] **Persona switching changes voice** — switch from Nova (Charon) to Atlas (Kore), voice is audibly ifferent within 1-2 seconds
- [ ] **GenUI renders during voice** — ask about stock data, chart appears while agent speaks. Not after. )URING
- [ ] **Cross-client action works** — trigger action from mobile → result visible on web dashboard within 3 econds
- [ ] **MCP plugin toggle** — enable Brave Search → agent performs web search → disable → agent falls ack gracefully
- [ ] **Mobile camera vision** — point at an object → agent describes it correctly via voice
- [ ] **No dead silences during tool calls** — agent gives verbal acknowledgment before every tool call
- [ ] **Loading states visible** — GenUI skeleton/shimmer shown during processing, not a blank screen
- [ ] **Dark mode polished** — every page, every component, every state looks correct in dark mode

## Stage 2 — Technical Implementation (30%) Verification

*"If I reviewed the code, would I give this a 5/5?"*

- [ ] **ADK usage is deep, not superficial** — `run_live()`, `McpToolset`, `sub_agents`, `RunConfig`, `SessionService`, Callbacks all present in code
- [ ] **Multi-agent hierarchy exists** — root_agent.py shows sub_agents, persona delegation via `transfer_to_agent`
- [ ] **Session persists across restart** — deploy to Cloud Run → chat → redeploy → resume previous ession
- [ ] **Error handling is implemented** — `before_model_callback`, `after_model_callback`, `on_tool_error_callback` present and functional
- [ ] **Grounding is verifiable** — ask a factual question → agent uses search tool → cites source in esponse
- [ ] **Context Compression enabled** — `RunConfig` includes `context_window_compression=True`
- [ ] **No hardcoded secrets** — all API keys in env vars or Secret Manager. `.env.example` provided, not `.env`
- [ ] **Firestore has multiple collections** — sessions, personas, mcp_configs, chat_history visible in irebase console
- [ ] **Cloud Logging works** — requests visible in Google Cloud Log Explorer
- [ ] **README setup works** — fresh `git clone` → follow README → app runs locally within 10 minutes

## Stage 2 — Demo & Presentation (30%) Verification

*"If I watched this 4-minute video, would I give this a 5/5?"*

- [ ] **Problem is clear by 0:30** — someone unfamiliar understands what's wrong with current AI ssistants
- [ ] **Working software shown by 1:00** — not slides, not architecture, ACTUAL software with user nteraction
- [ ] **At least one "wow" moment** — viewer pauses or considers rewatching (cross-client action is our est candidate)
- [ ] **Architecture diagram shown** — professional quality, all components visible, Google Cloud services abeled
- [ ] **Cloud deployment proven** — Cloud Run console, Firestore data, or Cloud Logging visible in video
- [ ] **Audio quality is excellent** — clear voice, no background noise, balanced volume between narration nd agent audio
- [ ] **Screen recording is readable** — text on screen is legible at 1080p playback. Font sizes adequate
- [ ] **Video ends strong** — impactful closing statement about the vision/future, not just "thanks for vatching"
- [ ] **Under 4 minutes** — time video in editor. Not 4:01

## Bonus Points Verification

- [ ] **Blog post published** — live URL on dev.to or Medium. Contains: problem statement, architecture iagram, code snippets, learnings. Mentions it's for the hackathon. #GeminiLiveAgentChallenge on social nedia post
- [ ] **Automated deployment** — `deploy/` directory contains Terraform files AND/OR deploy scripts. unning `terraform apply` or `./deploy.sh` actually deploys
- [ ] **GDG membership** — public Google Developer Group profile URL. Linked in Devpost submission

## Final Pre-Submit Sanity Check

- [ ] **Devpost submission is complete** — all 7 required fields filled
- [ ] **GitHub repo is PUBLIC** — test in incognito browser
- [ ] **YouTube video is PUBLIC** — test in incognito browser
- [ ] **Cloud Run service is RUNNING** — test the URL from a different network
- [ ] **All team members listed** — everyone on Devpost
- [ ] **No changes after deadline** — final commit is before March 17, 2026
- [ ] **Read the submission one more time** — typos, broken links, missing info